



NEOXAM DATAHUB - DATA DICTIONARY CUSTOMIZATION GUIDE

CORE ADMIN

DISCLAIMER – NEOXAM SOFTWARE DOCUMENTATION

NeoXam Portfolio Management SAS, a 'société par actions simplifiée' with share capital €15,575,931, registered with the Paris Company's register under No. 349 467 449 ('**NeoXam**'), is the owner of this Software Documentation. NeoXam holds all the rights required for the publication, together with, but not limited to, all logos, trademarks, screenshots, any data used herein. The Client is granted a right to use this Software Documentation only as agreed in writing with NeoXam. This Software Documentation is confidential, and the Client shall protect the confidentiality and NeoXam's intellectual property rights. Unless otherwise agreed in writing by NeoXam, this Software Documentation is provided 'as is' without warranty of any kind, either express or implied, including, but not limited to, that the Software Documentation will not include technical or other mistakes, inaccuracies or typographical errors. The Software Documentation may be changed, and NeoXam makes no commitment to update such materials, unless otherwise agreed in writing. Screenshots and examples provided in this Software Documentation are not contractual and may be changed, according to the Client's needs, operational and technical constraints. NeoXam assumes no responsibility for errors or omissions in the Software Documentation. In no event shall NeoXam be liable for any special, punitive, incidental, indirect or consequential damages of any kind, or any damages whatsoever, including, without limitation, those resulting from loss of use, data or profits, whether or not NeoXam has been advised of the possibility of such damages, and on any theory of liability, arising out of or in connection with the use of this Software Documentation.



TABLE OF CONTENTS

Introduction	1
Objects Internal Representation	1
Maintenance Main Guidelines	1
Modifying the Integrity Constraints	2
NeoXam DataHub Data Model	3
Data Dictionary Screen	3
Objects	3
Introduction	3
Type of Tables	4
MxClassificationItem	5
Objects Categories	8
Tables	9
Security Table	9
Creating Tables	10
Labelling	11
Fields	11
Field Type	12
Constraints and Uniqueness	13
Fields Association	16
Secondary References	17
4-Eye Validation Workflow	20
Manual Override	22
Code Historicization	24
Data Dictionary Setup	24
Screens	25
Query Screen	25
Detail Screen	28
Import	29
Identification	29
Conversion Function	31
Export	32
Uniqueness Control	33
Time Series Historization	34



Object	34
Configuration	35
History Discontinuity	37
Data Migration	38
Intraday Data	39
Data Dictionary	39
History Configuration	39
Time Zone	40
Displaying the “datetime” Field (delete or move down)	41
Configuration	42
Database Time Zone Settings	45
Date Time and Time Zone Manipulation	46
Data Migration	46
Data Masking	52
Introduction to Data Masking	52
Configuring Data Masking	54
Masking Functions	54
User-Level Restrictions	58
Class-Level Restrictions	59
Using Data Masking	61
Masking Securities Sedol (UK) Codes for a Specific User	61
Masking Interfaces	63
Data Masking and Scripting Languages	63

TABLE OF FIGURES

Figure 1 - MxClassificationItem example	6
Figure 2 - Defining a category of object consists in defining a dynamic condition on the object	9
Figure 3 - The "General" tab of the data dictionary	10
Figure 4 - References to classes	12
Figure 5 - Setting secondary references	17
Figure 6 – Configuring a detail screen to display the benchmark component security Sedol codes	18
Figure 7 - Displaying the SEDOL code with a second reference setup	19
Figure 8 - Validating or rejecting changes on fields with masked data	21
Figure 9 - Trying to apply a validation or rejection action on masked data	21
Figure 10 - Defining a strategy at the class level	23
Figure 11 - Configuring the HISTO table	24
Figure 12 - "Optimization" tab: Historicizing	25
Figure 13 - Searching identifiers by current date, a specific date or a relative date	26
Figure 14 - Looking for an ISIN code among the ISIN codes on day - 1	27
Figure 15 - Looking for an ISIN code among the ISIN codes on day - 2	28
Figure 16 – Sorting identifiers by type and date of validity	29
Figure 17 - Importing on the current date	30
Figure 18 - Specifying the validity date of a research	31
Figure 19 – Using the “free 4” field in a research	32
Figure 20 - Exporting on the current date	32
Figure 21 - Specifying the validity date of a research for a history export	33
Figure 22 – Ensuring that an identifier is used only once during a specific period	34
Figure 23 - The history depth combo box	35
Figure 24 - Defining "DATE" type as a key	35
Figure 25 - Time series tables with history management	36
Figure 26 - Chaining records through the "End Date" field	37
Figure 27 – Activating History Discontinuity on a record	37
Figure 28 – S&P rating showing record discontinuity	38
Figure 29 - Time data in a new field	39
Figure 30 – Declaring the time field in the history tab of the data dictionary	40
Figure 31 - Serialization and deserialization for time conversion	41
Figure 32 - Displaying the "datetime" field	41
Figure 33 - Editing the "datetime" field	42
Figure 34 - Time zones in the "General" tab	43
Figure 35 - The "QUOTE_DATETIME" field	44
Figure 36 - Updating the historicization settings	44
Figure 37 - Same quotation captured at different times in different places	45

Figure 38 - Updating the "SYSTEM_FIELD_TYPE" list	48
Figure 39 - Updating the MxUser class	49
Figure 40 – Import Mapping; the "ASSIGNMENT_OPERATOR" field for using operators such as ">" and "<"	50
Figure 41 - Export Mapping; the "ASSIGNMENT_OPERATOR" field for using operators such as ">" and "<"	51
Figure 42 - Defining operators	52
Figure 43 - Dynamic Masking	53
Figure 44 - The Masking Functions	56
Figure 45 - Setting up a mask	59
Figure 46 - Defining masking functions at the field-level	60
Figure 47 - A masking function applied to a class or a table	61
Figure 48 - Masking the Sedol (UK) codes of Securities for a specific user (1)	62
Figure 49 - Masking the Sedol (UK) codes of Securities for a specific user (2)	62
Figure 50 - Masking Rules: Results	63
Figure 51 - Masked field and masked value in a script	64

Introduction

The Data Dictionary reflects NeoXam DataHub data model to store objects in the database.

Objects Internal Representation

Each object in NeoXam DataHub has its own internal representation once loaded from the database.

The adopted representation is 3 dimensional. An object is in fact a 3D array [table] [record] [field]:

1. X-axis, there are as many elements as tables in the object.
2. Y-axis, there are as many elements as records in a specific table.
3. Z-axis, there are as many elements as fields in a specific record.

Note: The first table always has a 1:1 cardinality.

Maintenance Main Guidelines

As the data dictionary controls the database via the NeoXam DataHub server, all the updates to the data dictionary happen in real time. In fact, the process is like object updates in the database. The data dictionary, with its definitions, is itself a class of objects. Consequently, the update operations carried out in the DBMS are published to all the connected clients.

However, there are some limitations concerning the server <-> DBMS update link. All the requests are passed on in real-time at the level of the structure of the physical database, except for the following cases:

- Deletions of tables and fields are effective in the data dictionary but must be manually applied in the database.
- It is not currently possible to change the technical names of a class, a table or a field. To do this, two manual operations must be carried out directly at the level of the DBMS: a modification to the data dictionary and a significant modification in the database. Generally, these localised jobs are carried out by the NeoXam consultants.
- It is not possible to modify the type of a field. However, it is possible to increase the size of a String field. Decreasing is not allowed. This operation must be performed in the same way as name changes.

- Character string fields' size should not exceed 255 by default. If you increase the size of a given field to a value greater than 255, an error message box displays the following message: "***Size of string fields should not exceed 255. Use text type for huge strings***". However, it is possible to disable this control by editing the [system.db.stringfields.nosizecheck](#) technical parameter.
- It is not possible to reassign a field to another table. This field must be deleted in the current table, and recreated in another table, as if it were a new field. Pay special attention to data transfer between the 2 fields.

To the exception of these limitations, all the other maintenance actions, including those that occur in 90% of cases, are operational and take effect in real-time. In particular:

- Creation of entities: Class of object, table and field
- Any modification to the modifiable attributes of these entities.

Modifying the Integrity Constraints

The integrity constraints form part of the modifiable attributes and are completely managed by the server. These constraints are of particular interest to the required fields, the keys and the uniqueness concept in the tables.

NeoXam DataHub allows modifying the constraints even if the situation does not permit it. Let's suppose that a given field is made compulsory. We authorise this function even if null fields already exist in the database. Consequently, the check will be triggered:

- For new objects, for which the field is not completed.
- For pre-existing objects, if we try to modify them and save them without initialising this field.

Consequently, when this type of action occurs in the database, it is important to change what already exists by assigning non-null values to all the objects concerned.

The same principle applies to the management of uniqueness. The configuration is allowed even if duplicate records exist after the new definition of the key for the table concerned. Therefore, a clean-up operation must be immediately carried out so that any inconsistency is removed from the database; otherwise, such data will cause erroneous processes.

NeoXam DataHub Data Model

Data Dictionary Screen

Located in the **Core Admin** directory in the main menu, the **Data Dictionary** screen manages the whole data dictionary.

The **Data Dictionary** screen includes the following tabs:

- **General:** This tab allows defining the general structure of the object classes and particularly all the tables that compose them.
- **Fields:** This tab lists the definitions of the table fields.
- **Categories:** This tab defines categories of objects for a data class.
- **Versioning:** This tab deals with the versioning of objects.
- **Override:** This tab deals with manual override.
- **Validation:** This tab deals with for 4-Eyes validation.
- **Packaging:** This tab deals with data packaging and transfer.
- **Optimization:** This tab defines how caches and indexes are managed.
- **Archive:** This tab deals with data purge and archiving.
- **Misc:** This tab deals with miscellaneous facilities.

Objects

Introduction

The data dictionary defines the classes of the application's business objects.

Each class of business object represents a model in terms of tables and fields. It is a sort of entity-relationship diagram, with the specific feature that it has a master table, with which a group of secondary tables is associated.

The master table defines the instances of the class; this means that a record in the table defines an object of the class. The other tables are linked to the master table with a cardinality of 0 - 1 to 0 -n. The advantage of secondary tables is that they classify the characteristics of the class according to the nature of the information and its multiplicity in relation to the class or the master table.

For example, for the class for the Security object, we will typically have a secondary table containing prices, quite simply because an instance of Security may contain an infinite number of

prices. Here, we clearly see that the obvious advantage of this secondary table is to satisfy the need to manage volumes, thus aiming to normalize the model.

On the other hand, the CDS table of cardinality 0 - 1 was created to receive the characteristics of this type of contracts. We could, however, have put this information at the level of the master table. But we considered that the specific nature of this information required modeling CDS by means of a dedicated table.

Type of Tables

In most cases, we find certain typical structures:

- **Chronological tables**, which are therefore dated. These contain analysis, financial or price data. As they naturally constitute a history list, they are dated, and have 0 - n cardinalities with the master table. Generally, as well as the data, we associate the concept of provider to obtain greater depth.
- **"Codes" tables (CODE)**, composed of the code and the type of code, allow defining an infinite number of codes. These are generally values coded in their codification system: ISIN, Sedol, Cusip, ISO, etc.
- **"Name" tables (NAME)** guarantee that objects are internationalized. Following the example of the "codes" tables, we find the name and its language of expression. Currently, French and English are defined as standards, but the structure is supposed to be able to receive an infinite number of languages, the only extra effort being their translation.
- The **"status" tables (STATUS)** manage the life cycle of objects and any type of data-validity status and can indicate the stages in the process of acquisition and enhancement of objects. An infinite number of status values may exist for an infinite number of types of status. For example, we may have:
 - Type of status *validity* for which the status value is *active* or *inactive*.
 - Type of status completeness: *initialized*, *enhanced*, *complete*, *validated*.

These two status lines allow managing two distinct versions of statuses in parallel, each having its own life. It is thus possible to avoid mixing processes that are naturally and mutually incoherent and cannot operate in a single system.

- The **"comment" tables (COMMENT)** allow recording general, non-structured information, such as comments or descriptions. They are generally stamped with a date, a user and a type of comment. As they are quite free, they allow voluminous information of the text type to be received.
- The **"classification" tables (CLASSIFICATION)** answer classification purposes and make it possible to categorize objects. For example, we find classifications for sectors of activity

or ratings. Knowing that there may be an infinite number of classifications, we have, like with the "codes" tables, a classification value in a classification group.

- The « **classification item** » **class** is designed to handle classification tables that evolve over time and allows bitemporalization. It also supports multiple hierarchical levels, and the management of parent-child relationships within the classification.

MxClassificationItem

Note: The CLASSIFICATION_ITEM_CUSTOMIZER rule used in the Display column, allows to filter the drop-down list of the parent and ensures that you pick up a classification item from the same classification only.

PATH: HOMEPAGE > DATA DICTIONARY > EDIT DATA DICTIONARY MxCLASSIFICATIONITEM

Edit Data Dictionary MxClassificationItem

Q Actions

Fields

10 Hidden columns 1 Groups 3 Sorts

Table	Key	Name	Short Name (En.)	Type	Size	Reference	List	Display	Comment (En.)
CLASSIFICATION_ITEM(1)		ACTIVE	Active?	Integer / Reference		SMARTLIST_ITEM	Yes/No		
CLASSIFICATION_ITEM		END_DATE	End Dt	Date					
CLASSIFICATION_ITEM		FREE1	Free 1	String	255				
CLASSIFICATION_ITEM		FREE2	Free 2	String	255				
CLASSIFICATION_ITEM		FREE3	Free 3	String	255				
CLASSIFICATION_ITEM		FREE4	Free 4	String	255				
CLASSIFICATION_ITEM		ITEM_LEVEL	Level	Integer / Reference		SMARTLIST_ITEM	Classification Level		
CLASSIFICATION_ITEM		PARENT	Parent	Integer / Reference		CLASSIFICATION_ITEM		RsClient.custom.MxRuleCustom&CLASSIFICATION_ITEM_CUSTOMIZER	
CLASSIFICATION_ITEM	Key Level 3	START_DATE	Start Dt	Date					
CLASSIFICATION_ITEM	Key Level 2	CODE	Code	String	50				
CLASSIFICATION_ITEM	Key Level 1	CLASSIFICATION	Classification	Integer / Reference		SMARTLIST_ITEM	Classification Type		
CLASSIFICATION_ITEM_HISTO(15)		HISTO_SIVERSION	Version	Integer / Reference					
CLASSIFICATION_ITEM_HISTO		HISTO_NOTIFICATION	Dr-Notif	Date					
CLASSIFICATION_ITEM_HISTO		HISTO_OBJECTVERSION	Version	Integer / Reference					
CLASSIFICATION_ITEM_HISTO		HISTO_STATUS	Status	Integer / Reference					
CLASSIFICATION_ITEM_HISTO		HISTO_TEXT	Value Long	Text					
CLASSIFICATION_ITEM_HISTO		HISTO_USER	User	Integer / Reference		SMARTUSER			
CLASSIFICATION_ITEM_HISTO		HISTO_VALID_USER	User Last	Integer / Reference		SMARTUSER			
CLASSIFICATION_ITEM_HISTO		HISTO_VALIDATION	4-Eyes	Integer / Reference		SMARTLIST_ITEM	Validation Statuses		
CLASSIFICATION_ITEM_HISTO		HISTO_VALIDATION_ID	Validation	Integer / Reference					
CLASSIFICATION_ITEM_HISTO		HISTO_VALUE	Value Short	String	100				
CLASSIFICATION_ITEM_HISTO		HISTO_WORKFLOW	Workflow	Integer / Reference		FEEDBACK			
CLASSIFICATION_ITEM_HISTO	Key Level 4	HISTO_FIELD	Updated Field	String	40				
CLASSIFICATION_ITEM_HISTO	Key Level 3	HISTO_REC_ID	Record Updated	Integer / Reference					
CLASSIFICATION_ITEM_HISTO	Key Level 2	HISTO_DATE	Dr-Date	Date					
CLASSIFICATION_ITEM_HISTO	Key Level 1	HISTO_TABLE	Updated Table	String	40				
CLASSIFICATION_ITEM_LINK(2)		CLASSIFICATION_LINK_TYPE	Classif Link from	Integer / Reference		CLASSIFICATION_ITEM		RsClient.custom.MultiReferenceCustom&CLASSIFICATION_LINK_TYPE	
CLASSIFICATION_ITEM_LINK	Key Level 1	CLASSIFICATION_LINK_TYPE	Classif Link Type	Integer / Reference		SMARTLIST_ITEM	Classification Item Link Type		
CLASSIFICATION_ITEM_NAME(2)		NAME	Name	String	100				
CLASSIFICATION_ITEM_NAME	Key Level 1	LANGUAGE	Language	Integer / Reference		SMARTLIST_ITEM	Language		
CLASSIFICATION_ITEM_STATUS(2)		STATUS	Status	Integer / Reference		SMARTLIST_ITEM			
CLASSIFICATION_ITEM_STATUS	Key Level 1	STATUS_TYPE	Status Type	Integer / Reference		SMARTLIST_ITEM	Classification Type Status	RsClient.custom.MultiIntCustom&MxClassificationItem_STATUS_TYPE	Displays the value of the concerned status (eg Active, Cancelled, Inused...) Classification Item Status Type

MxClassificationItem uses the following lists:

- Classification Type
- Classification Level
- Classification Item Link Type
- Classification Type Status

GICS Classification Example

Let's see how to configure the GICS Classification, the **Classification Item - GICS Code** (GICS_CLASSIFICATION_CODE_Q) allows displaying classifications based on the classification type GICS group:

PATH: HOMEPAGE > CLASSIFICATION ITEM - GICS CODE

Classification Item - GICS Code

Query ▾

Actions ▾

2 Hidden columns Group 3 Sorts

Code	GICS Classification Label	Start Dt	End Dt	Level	Parent Code	Parent Label
10	Energy	01/01/1999	12/31/1999	Level 1		
15	Materials	01/01/1999	12/31/1999	Level 1		
20	Industrials test1	01/01/1999	12/31/1999	Level 1		
25	Consumer Discretionary	01/01/1999	12/31/1999	Level 1		
30	Consumer Staples	01/01/1999	12/31/1999	Level 1		
35	Health Care	01/01/1999	12/31/1999	Level 1		
40	Financials	01/01/1999	12/31/1999	Level 1		
45	Information Technology	01/01/1999	12/31/1999	Level 1		
50	Communication Services	12/01/2018	12/31/1999	Level 1		
55	Telecommunication Services-deprecated on 01/12/2018	01/01/1999	12/31/2018	Level 1		
60	Utilities	01/01/1999	12/31/1999	Level 1		
65	Real Estate	09/01/2016	12/31/1999	Level 1		
70	Multi Industry Fund	04/01/2023	12/31/1999	Level 1		
75	Blockchain	04/01/2023	12/31/1999	Level 1		
1010	Energy	01/01/1999	12/31/1999	Level 2	10	Energy
1510	Materials	01/01/1999	12/31/1999	Level 2	15	Materials
2010	Capital Goods	01/01/1999	12/31/1999	Level 2	20	Industrials test1
2020	Commercial & Professional Services	06/30/2008	12/31/1999	Level 2	20	Industrials test1
2030	Commercial Services & Supplies-deprecated on 30/08/2008	01/01/1999	08/30/2008	Level 2	20	Industrials test1
2040	Transportation	01/01/1999	12/31/1999	Level 2	20	Industrials test1
2510	Automobiles & Components	01/01/1999	12/31/1999	Level 2	25	Consumer Discretionary
2520	Consumer Durables & Apparel	01/01/1999	12/31/1999	Level 2	25	Consumer Discretionary
2530	Consumer Services	04/30/2005	12/31/1999	Level 2	25	Consumer Discretionary
2540	Hotels, Restaurants & Leisure-deprecated on 30/04/2005	01/01/1999	04/30/2005	Level 2	25	Consumer Discretionary
2550	Media-deprecated on 01/12/2018	01/01/1999	12/01/2018	Level 2	25	Consumer Discretionary
2560	Consumer Discretionary Distribution & Retail	06/01/2023	12/31/1999	Level 2	25	Consumer Discretionary
2570	Retailing-deprecated on 01/06/2023	01/01/1999	06/01/2023	Level 2	25	Consumer Discretionary
3010	Consumer Staples Distribution & Retail	06/01/2023	12/31/1999	Level 2	30	Consumer Staples
3020	Food & Staples Retailing-deprecated on 01/06/2023	05/01/2003	06/01/2023	Level 2	30	Consumer Staples
3030	Food & Drug Retailing-deprecated on 01/06/2023	01/01/1999	05/01/2003	Level 2	30	Consumer Staples
3040	Food, Beverage & Tobacco	01/01/1999	12/31/1999	Level 2	30	Consumer Staples
3050	Household & Personal Products	01/01/1999	12/31/1999	Level 2	30	Consumer Staples
3510	Health Care Equipment & Services	01/01/1999	12/31/1999	Level 2	35	Health Care
3520	Pharmaceuticals, Biotechnology & Life Sciences	04/29/2006	12/31/1999	Level 2	35	Health Care
3530	Pharmaceuticals & Biotechnology-deprecated on 29/04/2006	01/01/1999	04/29/2006	Level 2	35	Health Care
4010	Banks	01/01/1999	12/31/1999	Level 2	40	Financials
4020	Financial Services	06/01/2023	12/31/1999	Level 2	40	Financials
4030	Diversified Financials-deprecated on 01/06/2023	01/01/1999	06/01/2023	Level 2	40	Financials
4040	Insurance	01/01/1999	12/31/1999	Level 2	40	Financials
4050	Real Estate-deprecated on 01/09/2016	01/01/1999	09/01/2016	Level 2	40	Financials
4510	Software & Services	01/01/1999	12/31/1999	Level 2	45	Information Technology
4520	Technology Hardware & Equipment	01/01/1999	12/31/1999	Level 2	45	Information Technology
4530	Semiconductors & Semiconductor Equipment	05/01/2003	12/31/1999	Level 2	45	Information Technology
5010	Telecommunication Services	12/01/2018	12/31/1999	Level 2	50	Telecommunication Services-deprecated on 01/12/2018
5020	Telecommunication Services-deprecated on 01/12/2018	01/01/1999	12/01/2018	Level 2	50	Telecommunication Services-deprecated on 01/12/2018
5030	Media & Entertainment	12/01/2018	12/31/1999	Level 2	50	Telecommunication Services-deprecated on 01/12/2018
5040	Media & Entertainment-deprecated on 01/12/2018	09/29/2018	12/01/2018	Level 2	50	Telecommunication Services-deprecated on 01/12/2018
5510	Utilities	01/01/1999	12/31/1999	Level 2	55	Utilities
6010	Equity Real Estate Investment Trusts (REITs)	06/01/2023	12/31/1999	Level 2	60	Real Estate
6020	Real Estate-deprecated on 01/06/2023	09/01/2016	06/01/2023	Level 2	60	Real Estate
6030	Real Estate Management & Development	01/18/2023	12/31/1999	Level 2	60	Real Estate
8020	Fund	04/01/2023	12/31/1999	Level 2	80	Multi Industry Fund
9010	Cryptocurrencies	04/01/2023	12/31/1999	Level 2	90	Blockchain
9020	Subsidiaries	04/01/2023	12/31/1999	Level 2	90	Blockchain
101010	Oil, Gas & Consumable Fuels	01/01/1999	12/31/1999	Level 3	1010	Energy
101020	Oil & Gas-deprecated on 30/04/2005	04/30/2005	12/31/1999	Level 3	1010	Energy
151010	Chemicals	01/01/1999	12/31/1999	Level 3	1510	Materials

Figure 1 - MxClassificationItem example

Edit Classification Item

The Edit Classification Item (CLASSIFICATION_ITEM_D) allows editing the classifications:

PATH: HOMEPAGE > CLASSIFICATION ITEM - GICS CODE > EDIT CLASSIFICATION ITEM

Edit Classification Item

Q Actions

Definition

Classification
GICS Group

Name
Energy

Code
10

Start Dt
01/01/1999

End Dt
12/31/1999

Level
Level 1

Hierarchy

Parent Code
Parent

System

ID
11

Version
2

Last Upd By
Tahri, Maryam

Last Upd Dt
02/20/2025

Last Upd Tim
12:16:42

Classification Item objects can be used instead of elements from a list (here, SECTOR_GICS list).

For example, in the SECTOR_GICS list you can see that **BANK** has the internal code 401010:

PATH: HOMEPAGE > LISTS > LIST: SECTORS - GICS SECTOR_GICS

List: Sectors - GICS SECTOR_GICS

Actions

SECTOR_GICS

Values

Hide columns Group 3 Sorts

Sort	Label - English	Label - French	Label - Chinese	Internal Code	Active	Free 1	Free 2	Free 3
	Pharmaceuticals	Pharmaceuticals		352020	Yes			
	Life Sciences Tools & Services	Life Sciences Tools & Services		352030	Yes			
	Banks	Banks		401010	Yes			
	Thriffs & Mortgage Finance	Thriffs & Mortgage Finance		401020	Yes			
	Diversified Financial Services	Diversified Financial Services		402010	Yes			
	Consumer Finance	Consumer Finance		402020	Yes			
	Capital Markets	Capital Markets		402030	Yes			
	Mortgage Real Estate Investment Trusts (REITs)	Mortgage Real Estate Investment Trusts (REITs)		402040	Yes			
	Insurance	Insurance		403010	Yes			
	Real Estate -- Discontinued effective 04/28/2006	Real Estate -- Discontinued effective 04/28/2006		404010	Yes			
	Real Estate Investment Trusts (REITs)	Real Estate Investment Trusts (REITs)		404020	Yes			
	Real Estate Management & Development	Real Estate Management & Development		404030	Yes			
	Internet Software & Services	Internet Software & Services		451010	Yes			
	IT Services	IT Services		451020	Yes			
	Software	Software		451030	Yes			
	Communications Equipment	Communications Equipment		452010	Yes			
	Technology Hardware, Storage & Peripherals	Technology Hardware, Storage & Peripherals		452020	Yes			
	Electronic Equipment, Instruments & Components	Electronic Equipment, Instruments & Components		452030	Yes			
	Office Electronics	Office Electronics		452040	Yes			
	Semiconductor Equipment & Products -- Discontinued effective 04/30/2003.	Semiconductor Equipment & Products -- Discontinued effective 04/30/2003.		452050	Yes			
	Semiconductors & Semiconductor Equipment	Semiconductors & Semiconductor Equipment		453010	Yes			

In the **GICS Code Classification Item** list, the **Banks** is listed using the same internal code as in the **GICS SECTOR** List. You will then be able to refer to this classification item object instead of using the SECTOR_GICS list:

PATH: HOMEPAGE > CLASSIFICATION ITEM - GICS CODE

Classification Item - GICS Code

Query

Start Dt <= MM/DD/YYYY End Dt > MM/DD/YYYY Code = Level =

CLASSIFICATION_ITEM

Actions

2 Hidden columns Group 3 Sorts

Code	GICS Classification Label	Start Dt	End Dt	Level	Parent Code	Parent Label
351020	Health Care Providers & Services	01/01/1999	12/31/1999	Level 3		
351030	Health Care Technology	04/29/2006	12/31/1999	Level 3		
352010	Biotechnology	04/29/2006	12/31/1999	Level 3	3520	Pharmaceuticals & Biotechnology-deprecated on 29/04/2006
352010	Biotechnology-deprecated on 29/04/2006	01/01/1999	04/29/2006	Level 3	3520	Pharmaceuticals & Biotechnology-deprecated on 29/04/2006
352020	Pharmaceuticals	04/29/2006	12/31/1999	Level 3	3520	Pharmaceuticals & Biotechnology-deprecated on 29/04/2006
352020	Pharmaceuticals-deprecated on 29/04/2006	01/01/1999	04/29/2006	Level 3	3520	Pharmaceuticals & Biotechnology-deprecated on 29/04/2006
352030	Life Sciences Tools & Services	04/29/2006	12/31/1999	Level 3	3520	Pharmaceuticals & Biotechnology-deprecated on 29/04/2006
401010	Banks	03/01/2014	12/31/1999	Level 3		
401010	Commercial Banks-deprecated on 01/03/2014	05/01/2003	03/01/2014	Level 3		
401010	Banks-deprecated on 01/05/2003	01/01/1999	05/01/2003	Level 3		
401020	Thriffs & Mortgage Finance-deprecated on 01/06/2023	05/01/2003	06/01/2023	Level 3		

The classification **Banks** is used, for example, in the security **Malayan Bank Sp ADR Shs Sponsored American Depositary Receipt Repr 2 Shs**:

PATH: HOMEPAGE > SECURITY SCREENING > EQUITY DETAIL: MALAYAN BANKING RG REGISTERED SHs

Equity detail: Malayan Banking Rg Registered Shs Golden copy

Actions

Codes			Security classifications		
Code type	Code	Market code	Class Prov	Classification	Classif Start Date
Internal Sustainalytics ID WPK (Germany) INTERNAL_134555 EQ00752506 862153					
Barcelona Stock Exc(B1 - Equity,Index)(3) BBG Global ID BBG Unique ID Bloomberg Ticker BBG00JVC01V0 EQ0000000062270819 MAYMYR Barcelona Stock Exc(B1 - Equity,Index) Barcelona Stock Exc(B1 - Equity,Index) Barcelona Stock Exc(B1 - Equity,Index)					
BOERSE BERLIN(3) BBG Global ID BBG Unique ID Bloomberg Ticker BBG0000B11362 EQ0012566200001002 MYB BOERSE BERLIN BOERSE BERLIN BOERSE BERLIN					
BOERSE DUESSELDORF(3) BBG Global ID BBG Unique ID Bloomberg Ticker BBG0000B11157 EQ0012566200001002 MYB BOERSE DUESSELDORF BOERSE DUESSELDORF BOERSE DUESSELDORF					
BOERSE MUENCHEN(3)					

Class Prov	Classification	Classif Start Date
Decalog Instrument Type(2)	EQUITIES - ORDINARY	01/01/2024
Golden copy	EQUITIES - ORDINARY	01/01/2024
Initialization	EQUITIES - ORDINARY	01/01/2024
GICS Industry(2)	Banks	01/01/2024
Golden copy	Banks	01/01/2024
Initialization	Banks	01/01/2024
GICS Industry Group(2)	Banks	01/01/2024
Golden copy	Banks	01/01/2024
Initialization	Banks	01/01/2024
GICS Sector(2)	Diversified Banks	07/22/2024
Golden copy	Diversified Banks	07/22/2024
Refinitiv	Diversified Banks	07/22/2024
GICS Sub-Industry(2)	Diversified Banks	01/01/2024
Golden copy	Diversified Banks	01/01/2024
Initialization	Diversified Banks	01/01/2024
ICD Country(2)	Banks	01/01/2024
Golden copy	Banks	01/01/2024
Initialization	Banks	01/01/2024

You can also set up MxSecurity and MxAgent classification type lists to refer to **Classification items** instead of SmartList item.

Objects Categories

The "**Categories**" tab allows defining categories of objects that can limit the objects available in joints. The case is particularly relevant with third parties (AGENT class) for which we define as many classifications as we have types of third parties (broker, issuer, counterparty, etc.). For example, for the TRADE object, we may define attributes such as BROKER, COUNTERPARTY, CUSTODIAN, etc., and link them by reference to classes with suitable constraints.

Defining a category consists in defining a dynamic condition on the object, as illustrated by the example below.

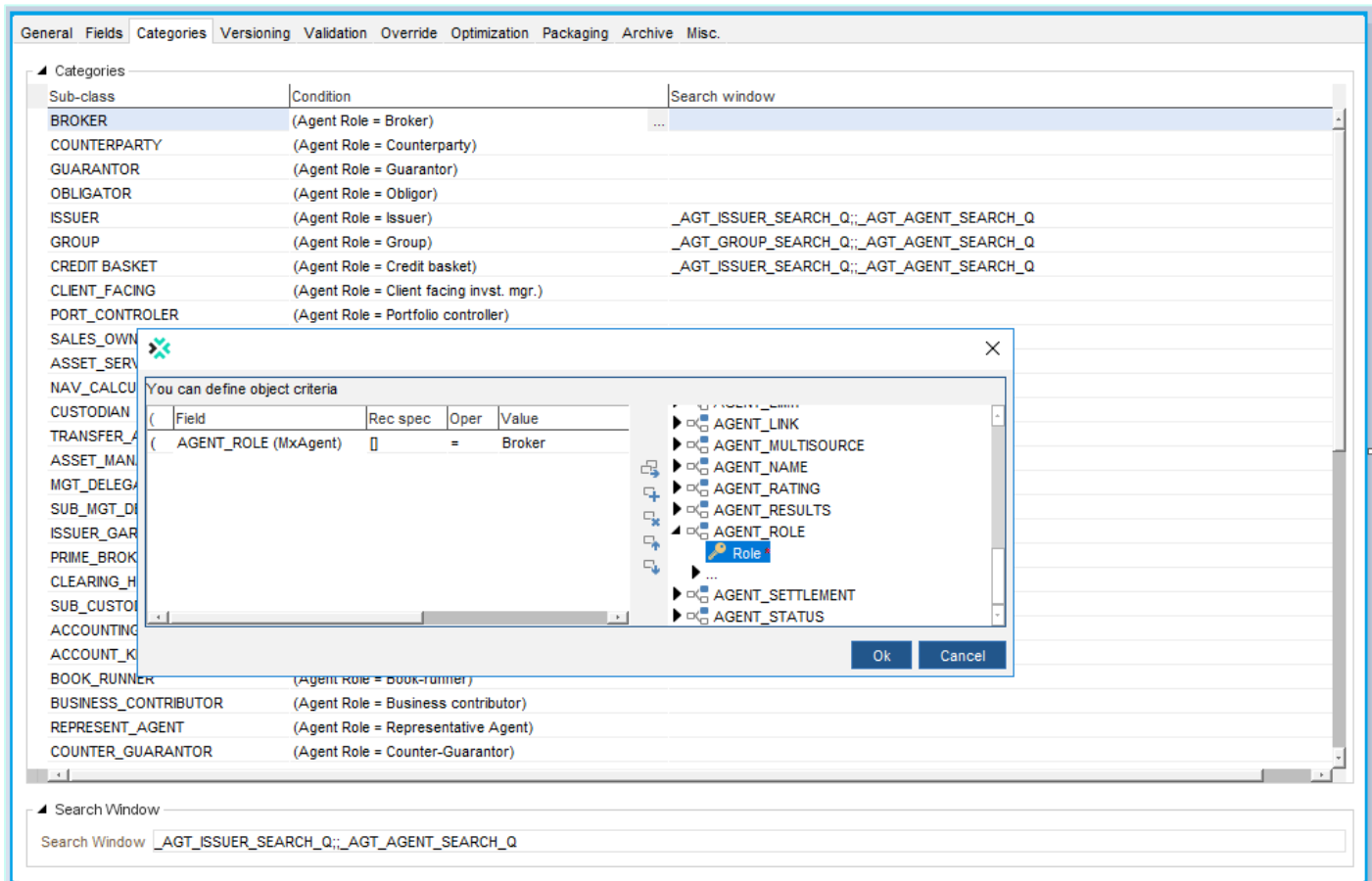


Figure 2 - Defining a category of object consists in defining a dynamic condition on the object

Tables

The framework of the object classes is defined by the creation of all the tables constituting the classes.

Security Table

The example below displays the model for the Security class, which is used to model all the securities and financial instruments, including currencies, indices and contracts.

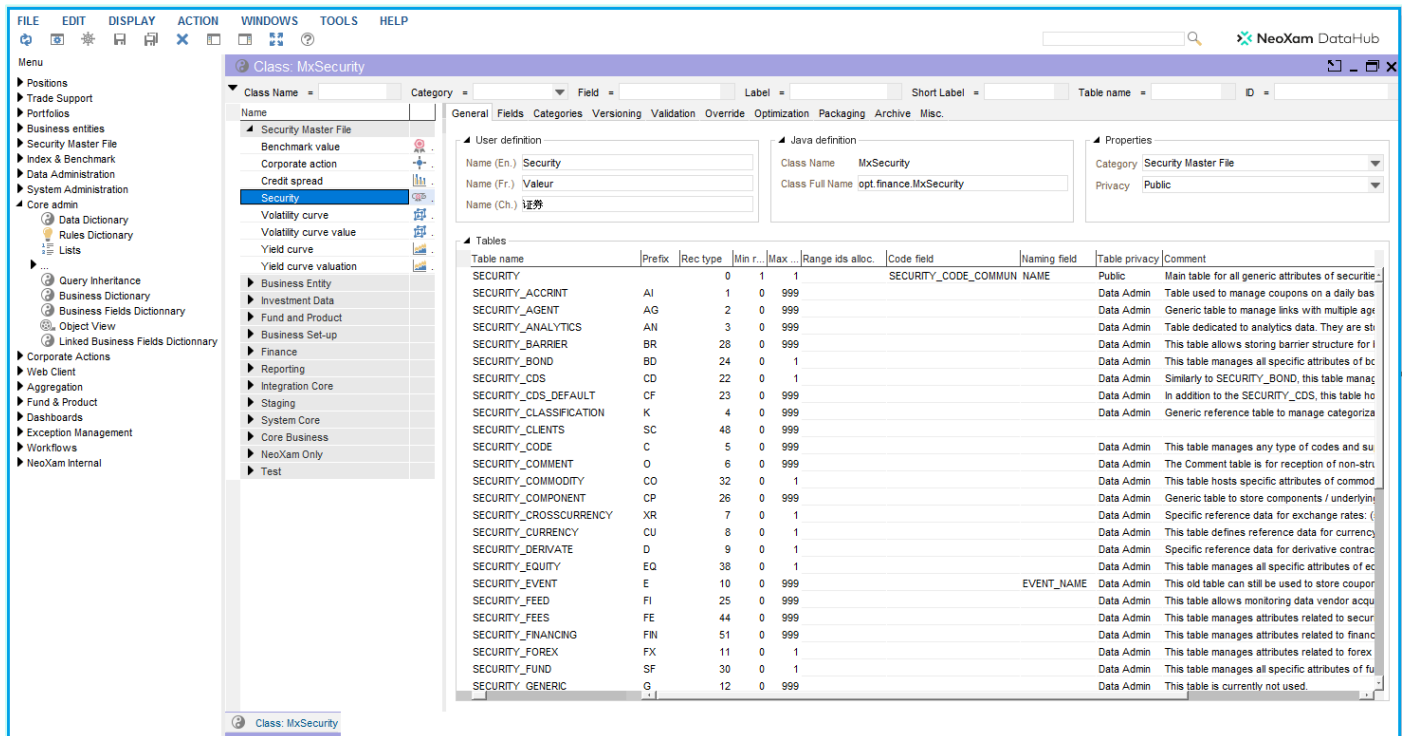


Figure 3 - The "General" tab of the data dictionary

This model includes the master table (SECURITY) and the group of secondary tables (SECURITY_XXX). Table names follow the convention below:

- The master table is used as a prefix for naming the secondary tables and must not contain any "_" characters (for example, we recommend YELDCURVE and not YIELD_CURVE).
- The names chosen must be significant and, as far as possible, must not be excessive abbreviations (SECURITY_CLASSIFICATION is better than SECURITY_CLASS or SECURITY_CLASSIF).
- Table names must not exceed 27 characters.

Creating Tables

The "tables" screen contains the following information:

- **Min:** Base value of cardinality (necessarily 0 or 1)
- **Max:** Upper value of cardinality (necessarily 1 or 999)
- **Naming field:** Indicates the field (of one of the tables) that appears in "lookups" to create joins between objects in the users' screens
- **Field code:** Indicates the field (of one of the tables) used as a pivot during data integration.

- **Privacy:** Indicates the level of ownership of the data, used in defining jurisdictions in relation to modeling data. Concerning the data dictionary, NeoXam is generally the owner.

Labelling

You can assign icons to classes of objects. They will be used by default in all the ad hoc graphical processes, in particular:

- in the graphical browser,
- in the shortcuts to screens in the main menu.

Fields

Once the tables of the object classes have been defined, the characteristics (or fields) of each one must be defined. The fields typically resemble those of the relational databases.

Consequently, we define their technical constraints and referential-integrity rules.

These fields usually include:

- **Name, short name** and **description**; each of them multilingual. The short name is the text that appears by default on screen. The long name is the information that pops up in the tool tip when you hover the cursor over the field in question. The purpose of the description is to document the data, as it is the case with the data dictionary.
- **Type** and **size** (of field); the following types are possible:
 - **Decimal**, for decimal numeric data
 - **Date**, for data of the "date" type
 - **String**, for character strings up to 255 characters by default. See the [Dictionary maintenance](#) paragraph for more details.
 - **Text**, for long data structures, with no apparent size limit.
 - **Integer / reference**, the former is for integers, the latter is used when the data is meant to point to another class of object. For example, in the Security class, there is the ISSUER field because this is data that has been modeled as an object class. It is associated with this class as a reference. To fields of this type can be affected values depending on Java limitations. They are ranged between Integer.MAX_VALUE and Integer.MIN_VALUE (see the following [Oracle documents](#)).
 - **Reference**: This is the reference to the object class (cf. above). The screen image below illustrates the concept of a related "issuer" and "portfolio" that point to the AGENT and PORTFOLIO classes.

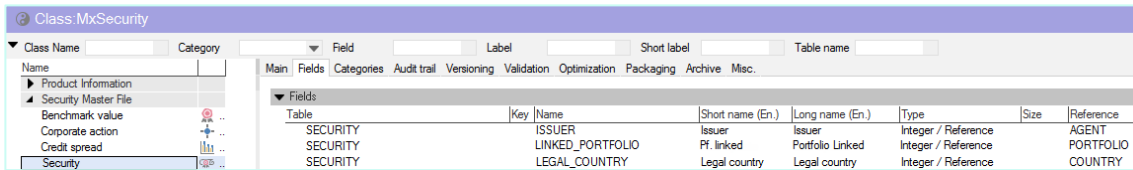


Table	Key	Name	Short name (En.)	Long name (En.)	Type	Size	Reference
SECURITY		ISSUER	Issuer	Issuer	Integer / Reference		AGENT
SECURITY		LINKED_PORTFOLIO	Pf. linked	Portfolio Linked	Integer / Reference		PORTFOLIO
SECURITY		LEGAL_COUNTRY	Legal country	Legal country	Integer / Reference		COUNTRY

Figure 4 - References to classes

- **List:** When a field can take a value from the list of possible values ([yes/no] or [Mr, Mrs, Ms]), a reference to the SMARTLIST class is added with the SMARTLIST_ITEM value. In the "List" field, provide the name of the list that defines the list of possible values.

Note: You can edit the list in question with the "Show List Detail" context-menu command. Right-click to get access to the command.

- **File:** fields with a type File can be added to any class, to store files or load files from the server on which NeoXam DataHub runs. The files are stored on the server in their original format. These fields take as value a file name or path.
- **Document:** fields with a Document type can also be added to any class. The only difference with the File type field is that Document stores or upload files from the database. The files are stored as BLOB (Binary Large Object) on the database. These fields take as value a file name or path.
- **Key:** Defines whether the data constitutes a key for the table. Other data in the table may also contribute to the key for the table (5 as a maximum).
- **Constraint:** This information allows defining a perimeter of possible values among all the objects available in the referenced class. In the case of the issuer, the AGENT class is referenced. However, this class defines all the third parties, the issuer being in one of their categories. It is better to define an issuer as a third-party with the role of issuer. To do this, in the Constraint field, add the ISSUER restriction. You can then deal with issuers only. The case is similar with currencies. The reference is SECURITY, and the constraint is CURRENCY.

Field Type

The dictionary defines the structure of each object known in the system and ensures data integrity (references, keys, mandatory fields, uniqueness ...).

Among the dictionary fields, we must pay attention to certain field types.

For instance:

Table	Field	Type	Reference	List
SECURITY	ISSUE_PRICE	Decimal		
SECURITY_NAME	NAME	String		
SECURITY_COMMENT	COMMENT_TEXT	Text		
SECURITY	CURRENCY	Integer / Reference	SECURITY	
SECURITY	ISSUE_DATE	Date		
SECURITY	QUANTITY_MODE	Integer / Reference	SMARTLIST_ITEM	Quantity mode
SECURITY		File		
SECURITY		Document		

- The CURRENCY field value is an ID to another object of type SECURITY.
- The ISSUE_DATE field value is in format dd/mm/yyyy.
- The QUANTITY_MODE field value is an ID of one of the elements in the SMARTLIST_ITEM (object SMARTLIST) table.
- The XY field is a file field type
- The XY field is a document field type

Mapping allows transferring data coming from one specific flow (for instance, from a file), of the string type, to one of the fields of an object in NeoXam DataHub, in the correct table and record, in the appropriate format.

Constraints and Uniqueness

Constraints are defined in the Categories tab in the Detail screen for the classes.

- **Mandatory:** The field of the object must be filled in. If it is not, the server rejects any attempt to save the object.
- **Index:** Relating to the database indexing mode (unique index, search index, for instance.)
- **Unique:** Determines how the uniqueness of data is managed within the table or among all the objects. This concept is extremely important and a whole chapter in the *Integration guides* document has been devoted to it with clear examples.
- **Precision:** Indicates the number of digits in a number.
- **Scale:** Indicates the number of digits to the right of the decimal point in a number.

- **#Digit:** Defines the number of decimals to display on screen. The decimals are truncated by default.
Caution: This parameter only has a visual effect and under no circumstances affects the internal storage of data.
- **Thousands:** Defines the separator for thousands in numeric data.
- **Display:** Defines the Java (or CustomUI) methods that cause specific visual behaviour on screen. This is the case with classifications, for which there is a group of classifications that points to a list. For each classification in the group, there is also a list that belongs to it. The role of the CustomUI is, therefore, to provide the link between the list of lists (groups of classifications) and the unitary lists (list of each of the groups).
- **Virtual and Script:** You can define the virtual fields used in the application; storage in the database is not required. Among them, there is the Computed option; it can execute the action defined (via a business rule) in the Script field. This action is executed each time it is used on screen; usually, after the screen has been refreshed. Consequently, when the field is not displayed, the business rule is not triggered. It is, therefore, not active when the object is manipulated through business rules or via the object-recovery APIs.
- **Mask function:** Allows you to define a data masking function for a dictionary field.
- **Mask parameter:** Allows you to define the character string that will be visible instead of the hidden data.

Please note that the 'Uniqueness' field in the dictionary specifies the table keys and can be of different types:

Type	Description
Intern	Ensures the uniqueness of the field among all the records in the table (unique key).
Extern	Ensures the uniqueness of the field among all objects.
Intern combined	Ensures the uniqueness of the field among all the records in the table (multiple keys).
Intern+Extern	Combines the previous 2 types.
Unique extern with same keys	The n-uplet constituting the keys must be unique whatever the object is.
Unique - extern, with same keys and in same time interval	Used only for history tables (e.g., SECURITY_CODE). The n-uplet constituting the keys must be unique whatever the object is during their validity period (from the start date to the end date).
Unique - intern in same time interval	Used only for history tables (e.g., SECURITY_CODE). Ensures the uniqueness of the field among all the records in the table (unique key) during its validity period (from the start date to the end date).

Example: the SECURITY_CODE table

Security_Code_Type	Security_Code
ISIN	FR000013110
Sedol	7309681

BNP PARIBAS SA

'Intern' on SECURITY_CODE_TYPE: Only one ISIN code is allowed per SECURITY object

SECURITY_CODE Table

Security_Code_Type	Security_Code
ISIN	FR000013110
ISIN	FR000000000

Wrong Object

SECURITY_CODE Table

Security_Code_Type	Security_Code
ISIN	FR000013110
Sedol	7309681

BNP PARIBAS SA

'Extern' on SECURITY_CODE: The ISIN code is unique to all the SECURITY objects, but a given SECURITY cannot have two ISIN codes.

SECURITY_CODE Table

Security_Code_Type	Security_Code
ISIN	FR0000121501
Sedol	7103526

PEUGEOT SA

SECURITY_CODE Table

Security_Code_Type	Security_Code
ISIN	FR0000131104
Sedol	7309681

BNP PARIBAS SA

'Intern' on SECURITY_CODE_TYPE and 'Extern' on SECURITY_CODE: Each SECURITY object owns a unique ISIN code, and no distinct SECURITY has the same ISIN code.

More examples:

- 'Intern' on SECURITY_NAME.NAME: Avoids having the same label in French and English for a given SECURITY.
- 'Extern' on SECURITY_NAME.NAME: Avoids having the same label, whatever the language is, for two distinct SECURITIES.
- 'Intern+Extern' on SECURITY_NAME.NAME: Avoids having the same label for two distinct SECURITY and for the same language.
- 'Extern with same keys' on SECURITY_CODE.SECURITY_CODE and Key level 1 on SECURITY_CODE.SECURITY_CODE_TYPE: Allows having more than one identical SECURITY_CODE, but only if their respective SECURITY_CODE_TYPE is different (ex: ISIN DE0000 and CUSIP DE0000).

Fields Association

Some fields may be linked to others and their values then depend on the values of those other fields.

For instance, the RATING_VALUE field in the SECURITY_RATING table depends on RATING_FEED and RATING_TYPE. The authorized values for RATING_VALUE then depend on the data feed (Bloomberg, Moody's, etc.) and on the rating type (short term, long term).

SECURITY_RATING is of the SMARTLIST_ITEM type, but the name of the associated list is dynamically defined in respect with the other fields.

To set the association up, use the DISPLAY property in the dictionary. The DISPLAY field makes reference to the name of the class created to define this association (for RATING_VALUE, it is 'opt.finance.clientui.SecurityRatingCustomUI').

Secondary References

The Secondary References field is used to refer to another class and retrieve codes from a referred object (target object). To retrieve the required code, it is necessary to specify the accurate keys values (e.g. to extract the ISIN code, you need to indicate the Market, Currency, and Country values).

Note: Key fields used to retrieve the required code (Market, Currency, Country, in the ISIN code example above) should be of the same type and table as the source field.

The secondary references field currently support the following two formats:

- Specifying only one source field that references the Market (EXCHANGE table only).
- Specifying multiple target fields by linking key fields of the source table to similar key fields in the referenced table.

Note: Secondary references can be set up for any field in the data dictionary.

Using Secondary References to refer to the Exchange table only

For example, below is the setup for displaying a security code referenced by a benchmark component that takes the benchmark component market into account.

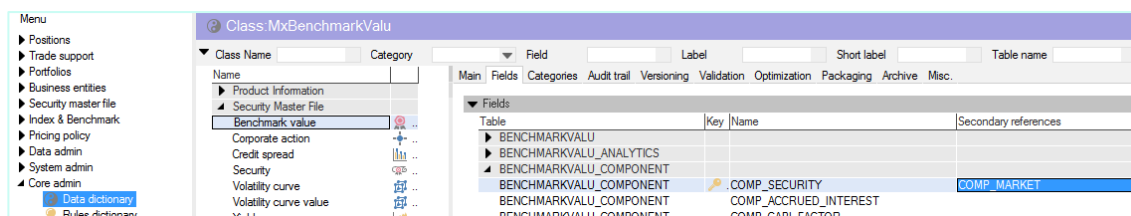


Figure 5 - Setting secondary references

Here, COMP_MARKET is the field name of the benchmark components; you can add other secondary references by adding another field name separated by spaces or commas.

For one benchmark value, you can display the benchmark component security Sedol codes; to do this, configure its detail screen, as below:

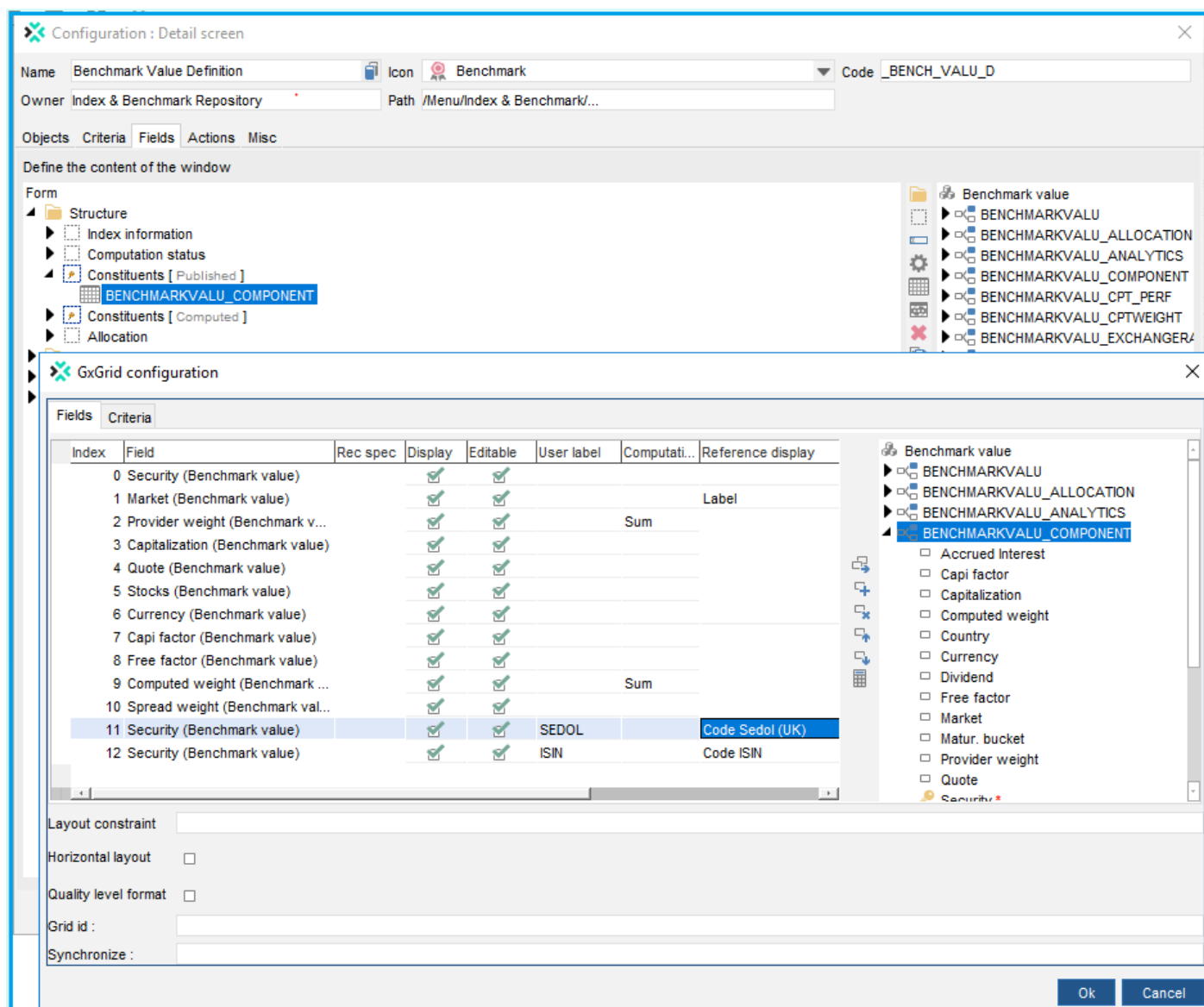


Figure 6 – Configuring a detail screen to display the benchmark component security Sedol codes

With the secondary reference setup this results in:

Benchmark Valuations

Value date from = 06/07/2020 to 16/08/2021 Benchmark = SX5P

Benchmark: SX5P

Structure Other details ...

Index information

Benchmark: SX5P Index information: Datastream

Publication type: Official Release Value date: 16/08/2021

Constituents

Security	Market	SEDOL	Provider weight
ABB LTD N (OME)	NORDIC GROWTH MARKET	7113815	

Equity detail: ABB LTD N (OME)

Overview Detail Ref. data IDs & classif. Markets Prices Charts Analytics Corp. action Audit ... Feeds Comp

Codes

Code type	Code	Market code	Code end date
Internal	INTERNAL_86223		31/12/2999
Sedol (UK)	7113815		31/12/2999
EUREX DEUTSCHLAND			
Internal	INTERNAL_86223_51	EUREX DEUTSCHLAND	31/12/2999
MIC - EUOTLX			
Internal	INTERNAL_86223_29	MIC - EUOTLX	31/12/2999
NORDIC GROWTH MARKET			
Datastream	698763	NORDIC GROWTH MARKET	31/12/2999
Internal	INTERNAL_86223_3	NORDIC GROWTH MARKET	31/12/2999
ISIN	CH0012221716	NORDIC GROWTH MARKET	31/12/2999
Sedol (UK)	7113815	NORDIC GROWTH MARKET	31/12/2999
SIX SWISS EXCHANGE			
Datastream	922924	SIX SWISS EXCHANGE	31/12/2999
Internal	INTERNAL_86223_4	SIX SWISS EXCHANGE	31/12/2999
ISIN	CH0012221716	SIX SWISS EXCHANGE	31/12/2999
Sedol (UK)	7108899	SIX SWISS EXCHANGE	31/12/2999

Figure 7 - Displaying the SEDOL code with a second reference setup

Here, several Sedol codes could be displayed for the "ABB LTD N (OME)" security, but the one defined in the Reference field is from the benchmark component with Nordic Growth Market as market must be 7113815.

Using Multiple links in the Secondary References Field

Specifying multiple links in the secondary references field, is specifically useful when the security is listed in the same exchange and currency, under multiple listings (e.g. multi-FIGI) and prices. The Secondary references of a field then allow you to specify multiple links between the key fields of the source table and similar key fields in the target table. The SECURITY_LEG table uses the following main fields to define underlying equities:

- LEG_SECURITY to identify the underlying security,
- LEG_SECURITY_CURRENCY to define the currency,
- LEG_SECURITY_EXCHANGE for the exchange.

Using the following pattern: `source_field1 = target_field1`, `source_field2=target_field2`, etc., you can specify a target field within the referenced table.

For example, the secondary field can contain:

`LEG_SECURITY_EXCHANGE = SECURITY_CODE_EXCHANGE, LEG_SECURITY_CURRENCY = SECURITY_CODE_CURRENCY`

4-Eye Validation Workflow

A user cannot validate or reject changes on fields with masked data:

Validation Panel

4Yeux = Waiting

Histo date from = 02/06/2021 to 02/06/2021

User = [User]

Last user = [User]

Sec type = [Sec type]

SECURITY_ID = [SECURITY_ID]

Table	Sel	Status	Record	Field	Update Dt	User	Prev Value	New Value	Apprvd By
Security									
0 Fce 22 Bds 0% Government...									
SECURITY	☒	Waiting	[SECURITY, 0 Fce 22 Bds 0% Government Bonds 2016-25.5.22]	Primary place	02/06/2021	Full access u...	****	****	
SECURITY	☒	Waiting	[SECURITY, 0 Fce 22 Bds 0% Government Bonds 2016-25.5.22]	Last Charac. ...	02/06/2021	Full access u...	18/05/2021	02/06/2021	

Overview Detail Ref. data IDs & classif. Markets Prices Charts Analytics Corp. action Audit ... Feeds Completeness

Identification

Name 0 Fce 22 Bds 0% Government Bonds 2016-25.5.22 ISIN FR0013219177 Ticker FRTR 0 05/25/22

Short name CUSIP AL2130345 RIC

CFI Code

Main

Sec main type Equity Sec type FI - Government

Issuer MxAgent#78799 Country France

Currency Euro Primary place ****

Lifecycle Active Availability Complete

4-Eyes 4-Eyes view Golden copy Current object

Approve Selected Reject Selected Select All Deselect All

Security: 0 Fce 22 Bds 0% Government Bonds 2016-25.5.22 Validation Panel

1 / 1 Security 1 line

Figure 8 - Validating or rejecting changes on fields with masked data

If a user tries to apply a validation or rejection action, a message pops up:

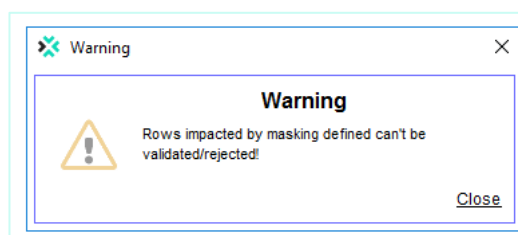


Figure 9 - Trying to apply a validation or rejection action on masked data

Manual Override

To manually override data, you can implement the following strategies:

- Cancel if new data vendor value
- Cancel only when expire (Keep new value)
- Cancel only when expire (Retrieve original value).
- Never expire
- Here is a summary of the possible values taken for each strategy:

Strategy	Value taken after the override
Cancel if new data vendor value	New data vendor
Cancel only when expire (Keep new value)	Any new data input after expiration.
Cancel only when expire (Retrieve original value)	Original value (before the override) until new data input after expiration.

You can configure the Manual Override strategies at the class, the table, and the field levels.

The priority rule is as follows:

- If the strategy is defined at the field level, the field is considered; the table or the class strategy is not considered.
- If the strategy is defined at the table level, the table is considered; the class strategy is not considered.
- If the strategy is defined at the class level, the class is considered.

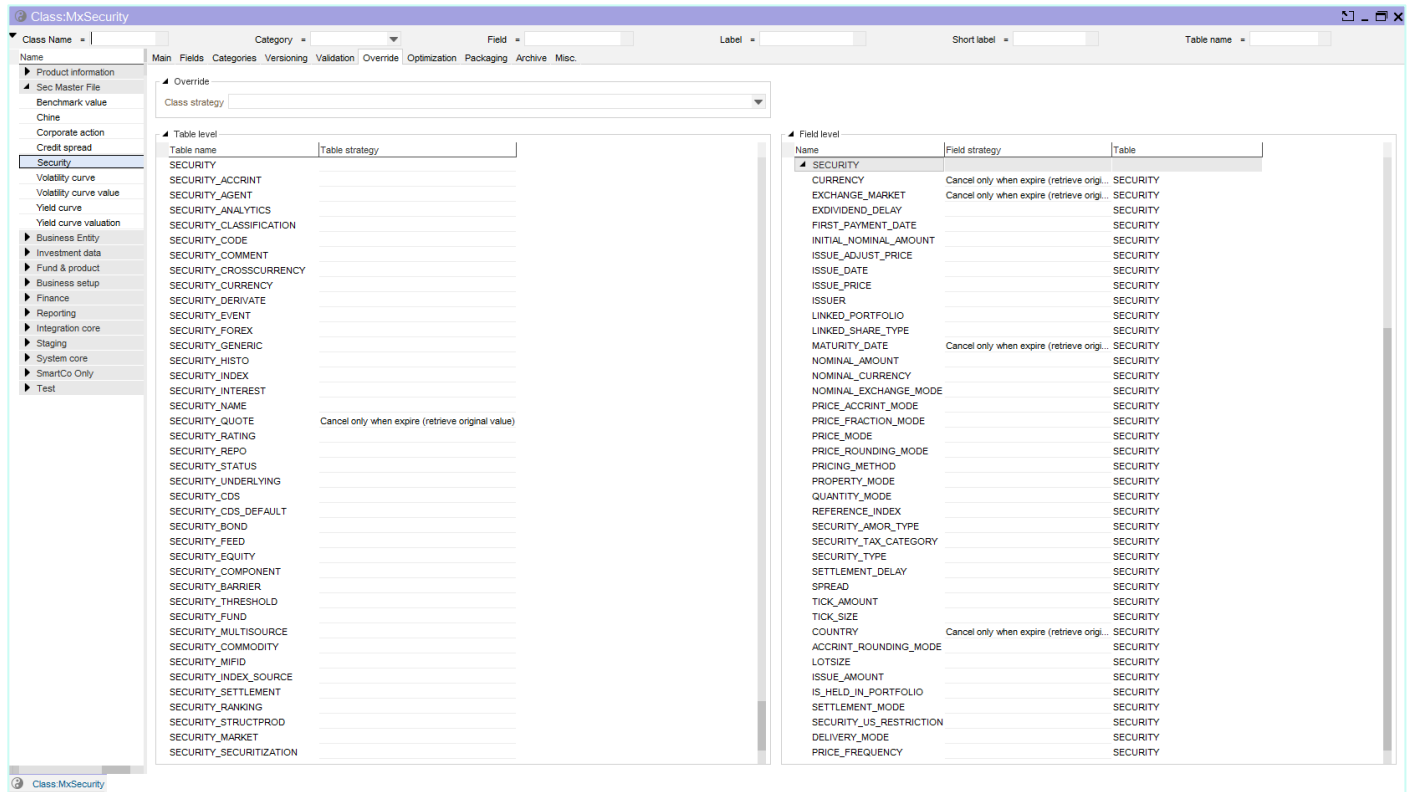


Figure 10 - Defining a strategy at the class level

Table setup:

Ensure that the *_HISTO table is configured as follows for the class you are applying manual override to:

Fields						
Table	Key	Name	Type	Size	Reference	List
AGENT_HISTORY		AGENT_CLASSIFICATION				
AGENT_HISTORY		AGENT_CLIENTS				
AGENT_HISTORY		AGENT_CODE				
AGENT_HISTORY		AGENT_CODE_COMMUN				
AGENT_HISTORY		AGENT_CODE_MULTIPLE				
AGENT_HISTORY		AGENT_COMMENT				
AGENT_HISTORY		AGENT_CORRESPONDANT				
AGENT_HISTORY		AGENT_COUNTRY				
AGENT_HISTORY		AGENT_ESTIMATE				
AGENT_HISTORY		AGENT_FEED				
AGENT_HISTORY		AGENT_FUND				
AGENT_HISTORY		AGENT_HISTO				
AGENT_HISTORY		HISTO_DATE	Date			
AGENT_HISTORY		HISTO_FIELD	String	40		
AGENT_HISTORY		HISTO_COMMENT	Text			
AGENT_HISTORY		HISTO_END_DATE	Date			
AGENT_HISTORY		HISTO_IS_OVERRIDE	Integer / Reference			
AGENT_HISTORY		HISTO_ISVERSION	Integer / Reference			
AGENT_HISTORY		HISTO_NOTIFICATION	Date			
AGENT_HISTORY		HISTO_OBJECTVERSION	Integer / Reference			
AGENT_HISTORY		HISTO_OVERRIDE_STATUS	Integer / Reference			
AGENT_HISTORY		HISTO_PROVIDER_VALUE	Text			
AGENT_HISTORY		HISTO_REC_ID	Integer / Reference			
AGENT_HISTORY		HISTO_STATUS	Integer / Reference			
AGENT_HISTORY		HISTO_TABLE	String	40		
AGENT_HISTORY		HISTO_TEXT	Text			
AGENT_HISTORY		HISTO_TIME	String	8		
AGENT_HISTORY		HISTO_USER	Integer / Reference		SMARTUSER	
AGENT_HISTORY		HISTO_VALID_USER	Integer / Reference		SMARTUSER	
AGENT_HISTORY		HISTO_VALIDATION	Integer / Reference		SMARTLIST_ITEM	Validation Statuses
AGENT_HISTORY		HISTO_VALIDATION_ID	Integer / Reference		VALIDATION	
AGENT_HISTORY		HISTO_VALUE	String	100		
AGENT_HISTORY		HISTO_VALUE_INIT	Text			
AGENT_HISTORY		HISTO_WORKFLOW	Integer / Reference		FEEDREJECT	

Figure 11 - Configuring the HISTO table

Code Historicization

NeoXam DataHub allows managing identification table as history tables with a start date and an end date of validity for each identifier.

Data Dictionary Setup

As for history record, you must create a start date and an end date in the identification table. This field must be specified in the optimization tab.

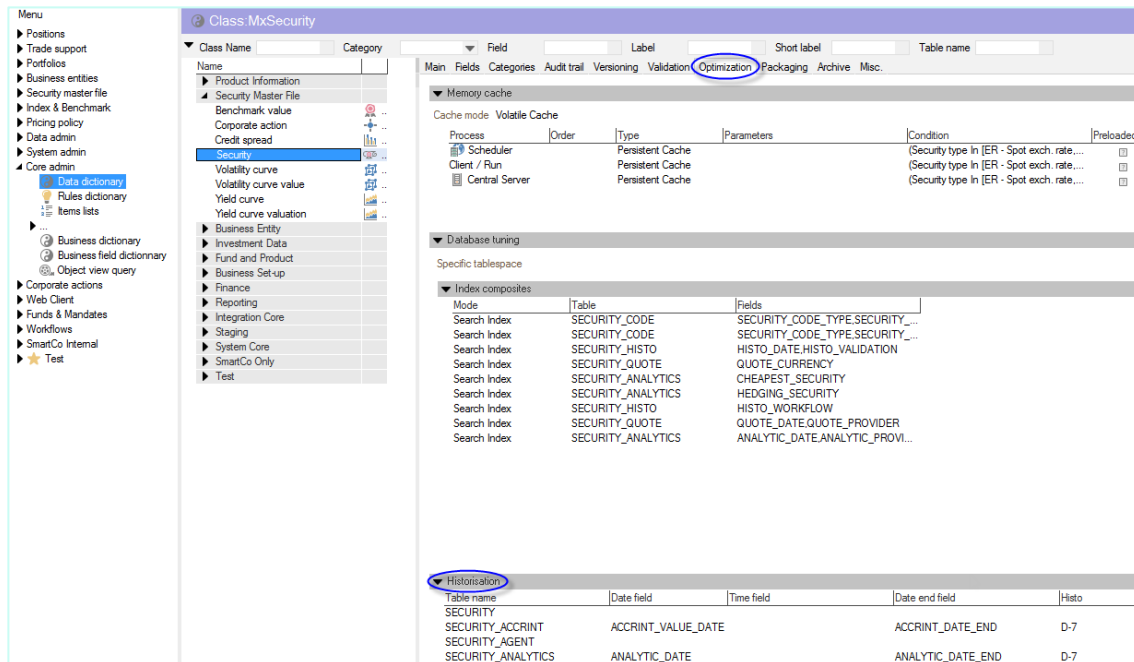


Figure 12 - "Optimization" tab: Historicizing

SQL example for initialization:

```
UPDATE SECURITY_CODE SET SECURITY_CODE_DATE = C_UPDATE_DATE,
SECURITY_CODE_DATE_END = '29991231'
```

Screens

Query Screen

In the query form, you can look for identifiers by current date, specific date or relative date.

In the result, you can see any identifier: the latest or the value on a date or a relative date.

For example, to see the last ISIN code, you must specify the end date “12/31/2999”:

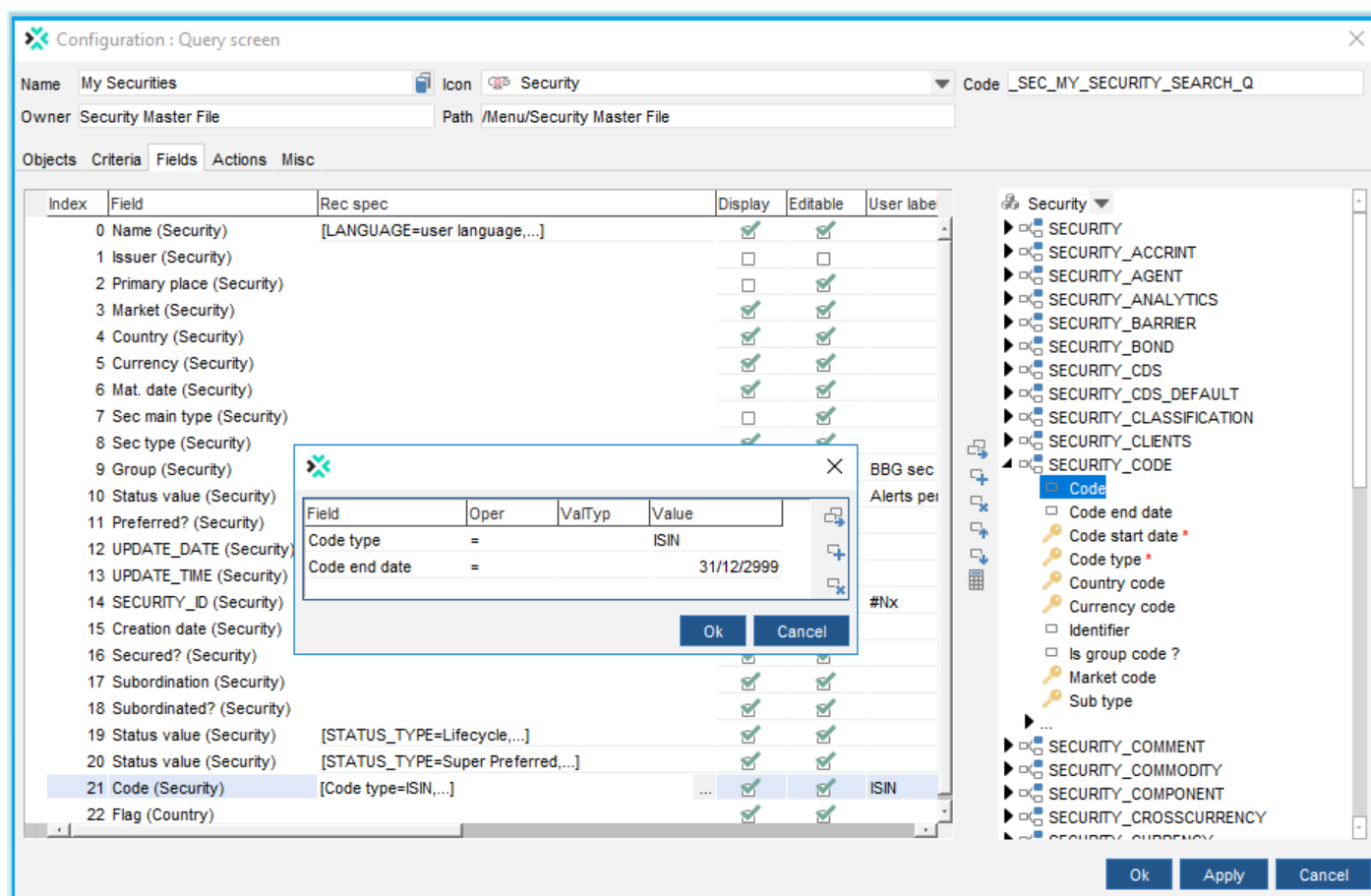


Figure 13 - Searching identifiers by current date, a specific date or a relative date

For example, to look for an ISIN code among the ISIN codes on day - 1, you must specify, in the record specification of the criteria, the start date with the new “value at date” operator on the relative date, “D-1”:

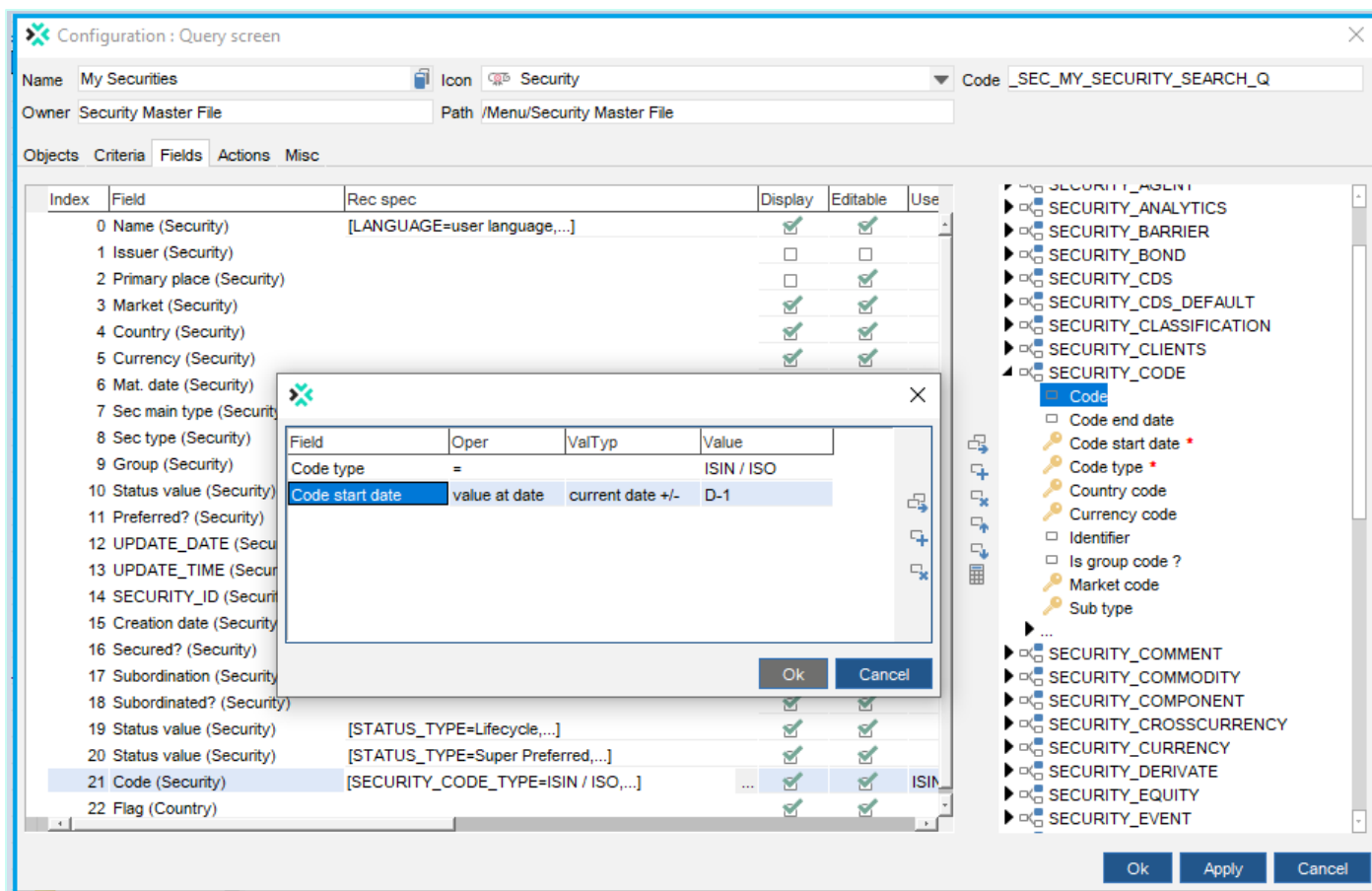


Figure 14 - Looking for an ISIN code among the ISIN codes on day - 1

For example, to see the ISIN code on day - 2, you must specify, in the record specification of the field, the start date with the new “value at date” operator on the relative date, “D-2”:

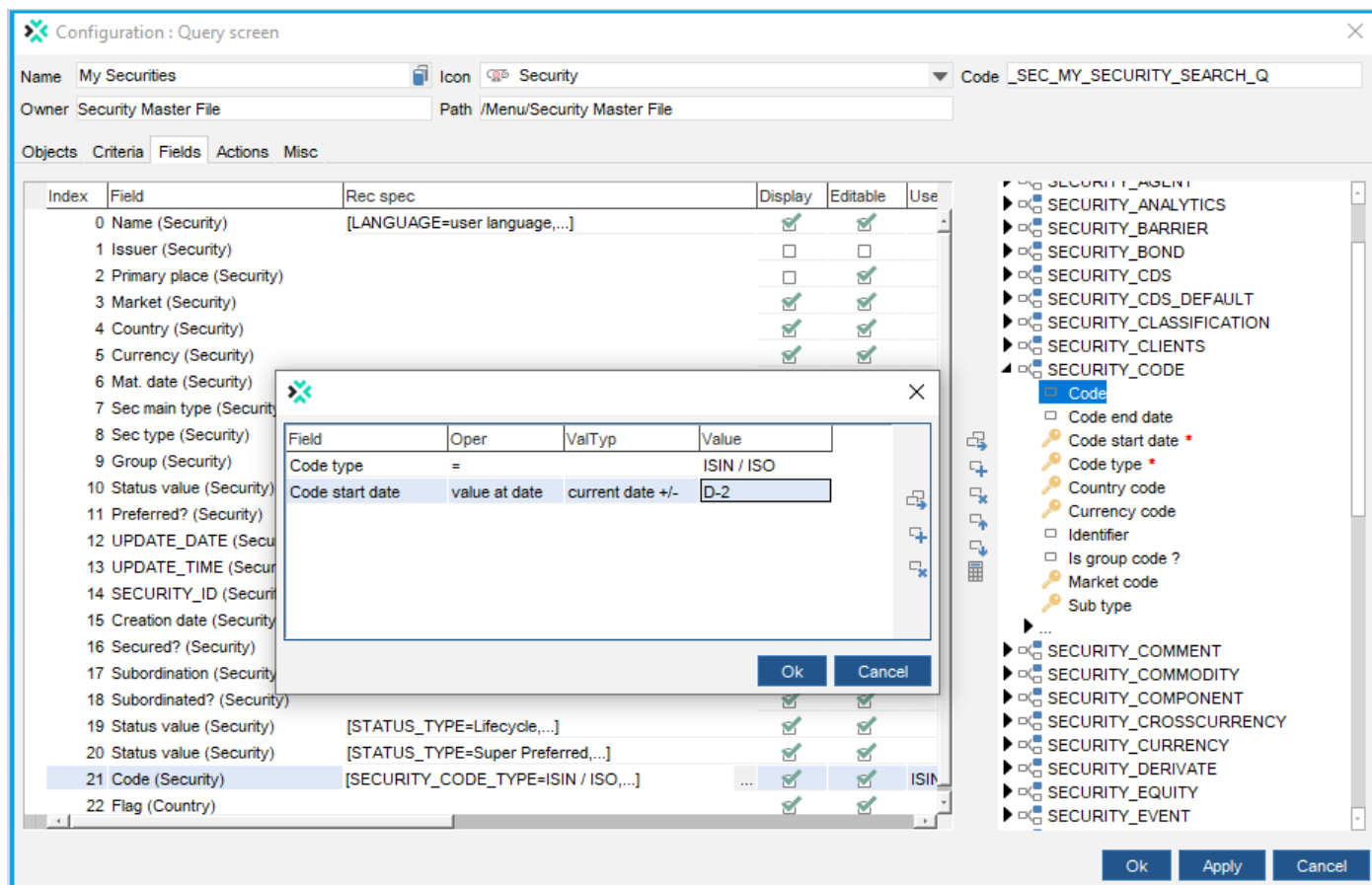


Figure 15 - Looking for an ISIN code among the ISIN codes on day - 2

Detail Screen

Like in any grid, you can see any kind of identifier sorted by type and date of validity.

Example:

[illegible]

Figure 16 – Sorting identifiers by type and date of validity

Note: To easily create a record in the identification table, initialize the start date with the current date through a business rule (type of trigger: "record created").

Import

Identification

To import on the current date, specify the end date: "31/12/2999".

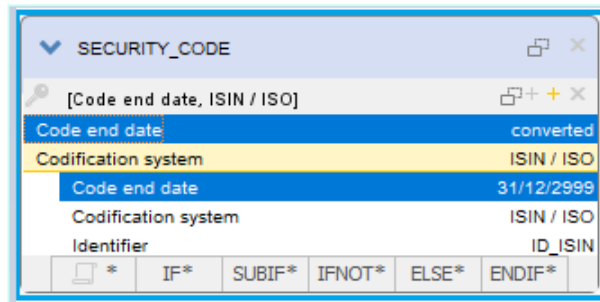


Figure 17 - Importing on the current date

For a history import, specify the date of validity of the research.

It will be converted in SQL into: “Start date <= date of integration and end date > date of integration”.

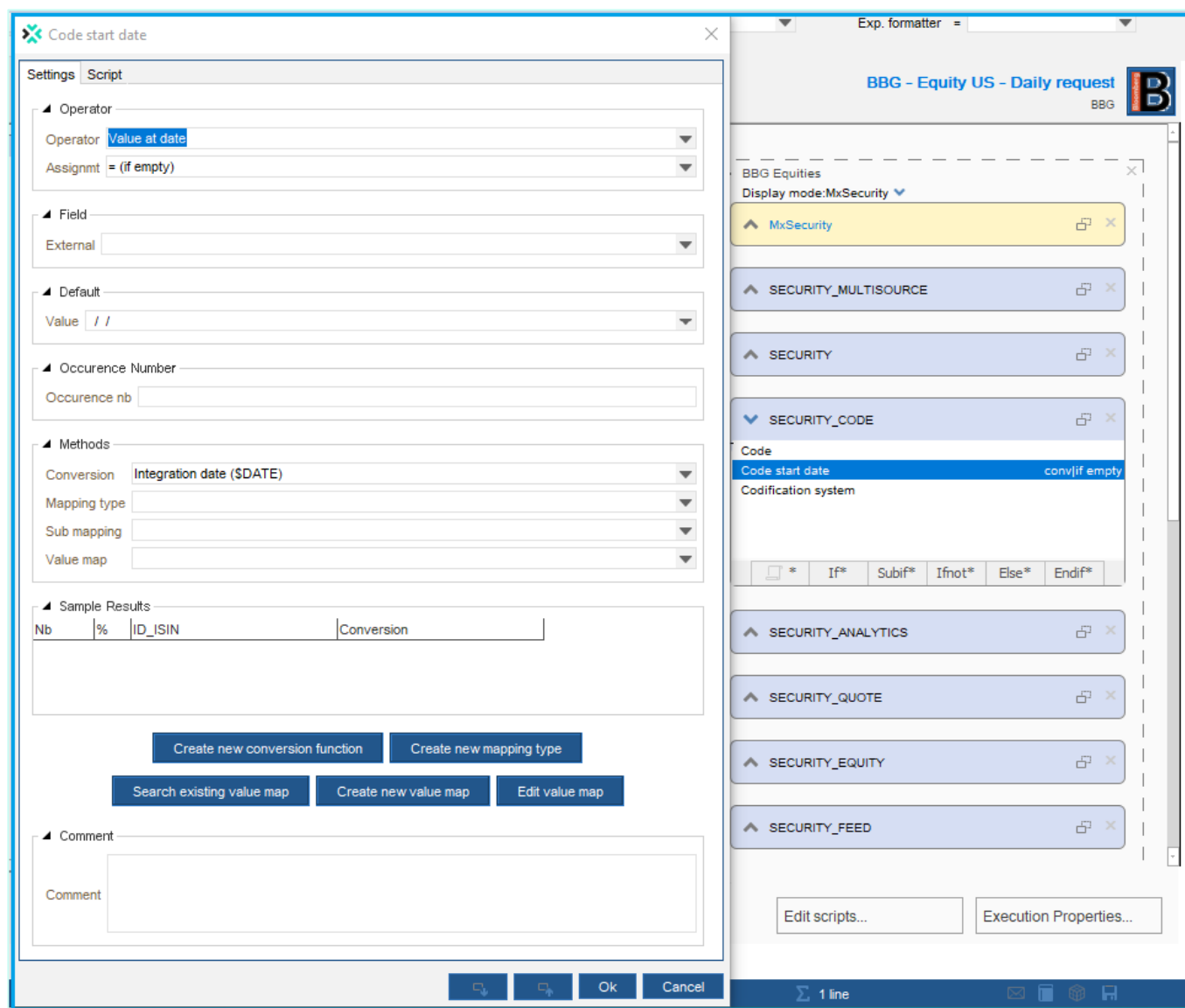


Figure 18 - Specifying the validity date of a research

Conversion Function

In the “free 4” field, you can give a new record specification for the research.

This function looks for an ISIN among the ISIN codes on the %DATE% date. The %DATE% is a property of the import. It is useful for history import. This function looks for an ISIN among the current ISIN codes.

Values

Definition details

Categories

Definitions

Name (En.)

External feed - Standard mappings

Name (Fr.)

Flux externe - Mappings standards

Code

XFEEDMAP_STANDARD_MAPPINGS

Type

Privacy

Root Admin

Custom Sort

☐

Free Flag

☐

Name (Ch.)

External feed - Standard mappings

Category

Connectivity

Type Complement

Feed

Not Hierarchical Sort

☐

Values

ID

NX #	Sort	Label - English	Label - French	Label - Chinese	Internal Code	Active	Free 1	Free 2	Free 3	Free 4	Item
38060		Benchmark by Capco Code	Benchmark by Cap...	Benchmark by Capco...	Benchmark by Capco C...	☒	MxSecurity	SECURITY_CODE	SECURITY_CODE_TYPE=CAPCO	SECURITY_CODE_DATE_END=29991231	R
435845		Benchmark byRimes Code	Benchmark byRime...	Benchmark by Rimes Co...	Benchmark byRimes Code	☒	MxSecurity	SECURITY_CODE	SECURITY_CODE_TYPE=RIMES	SECURITY_CODE_DATE_END=29991231	R
445451		Currency by ISIN	Devise par ISIN	Currency by ISIN	Currency by ISIN	☒	MxSecurity	SECURITY_CODE	SECURITY_CODE_TYPE=ISIN	SECURITY_CODE_DATE_END=29991231	R
23396		Currency by ISO	Currency by ISO	Currency by ISO	Currency by ISO	☒	MxSecurity	SECURITY_CODE	SECURITY_CODE_TYPE=ISO	SECURITY_CODE_DATE_END=29991231	R
3656		Currency by ISO3	Devise par ISO3	Currency by ISO3	Currency by ISO3	☒	MxSecurity	SECURITY_CODE	SECURITY_CODE_TYPE=ISO	SECURITY_CODE_DATE_END=29991231	R
445452		Currency by Numeric	Devise par Numeric	Currency by Numeric	Currency by Numeric	☒	MxSecurity	SECURITY_CODE	SECURITY_CODE_TYPE=UN	SECURITY_CODE_DATE_END=29991231	R
445404		Currency by TK code	Currency by TK co...	Currency by TK code	Currency by TK code	☒	MxSecurity	SECURITY_CODE	SECURITY_CODE_TYPE=TK	SECURITY_CODE_DATE_END=29991231	R
552077		Currency by WM Daten c...	Devise par WM Dat...	Currency by WM Daten ...	Currency by WM Daten ...	☒	MxSecurity	SECURITY_CODE	SECURITY_CODE_TYPE=WKN	SECURITY_CODE_DATE_END=29991231	D
615621		Security - by Internal Co...	Security - by Intern...	Security - by Internal ...	Security - by Internal Co...	☒	MxSecurity	SECURITY_CODE_COMMUN	SECURITY_CODE_TYPE=INTERNAL	SECURITY_CODE_DATE_END=29991231	D
498443		Security - by TK HÄndle...	Security By TK HÄ...	Security By TK HÄndle...	Security By TK HÄndle...	☒	MxSecurity	SECURITY_CODE_COMMUN	SECURITY_CODE_TYPE=TKH	SECURITY_CODE_DATE_END=29991231	R
20982		Security by CAF ID	Security by CAF ID	Security by CAF ID	Security by CAF ID	☒	MxSecurity	SECURITY_CODE	SECURITY_CODE_TYPE=CAF	SECURITY_CODE_DATE_END=29991231	R
498451		Security by China Sec Co...	Security by China ...	Security by China Se...	Security by China Sec C...	☒	MxSecurity	SECURITY_CODE_COMMUN	SECURITY_CODE_TYPE=CNSECCO...	SECURITY_CODE_DATE_END=29991231	R
445435		Security by CINS code	Security by CINS c...	Security by CINS code	Security by CINS code	☒	MxSecurity	SECURITY_CODE_COMMUN	SECURITY_CODE_TYPE=U*	SECURITY_CODE_DATE_END=29991231	R
52009		Security by code Valor	Security by code V...	Security by code Valor	Security by code Valor	☒	MxSecurity	SECURITY_CODE_COMMUN	SECURITY_CODE_TYPE=CH	SECURITY_CODE_DATE_END=29991231	R
38061		Security by CUSIP code	Security by CUSIP ...	Security by CUSIP co...	Security by CUSIP code	☒	MxSecurity	SECURITY_CODE_COMMUN	SECURITY_CODE_TYPE=US	SECURITY_CODE_DATE_END=29991231	R
7615		Security by CUSIP ID	Security by CUSIP ID	Security by CUSIP ID	Security by CUSIP ID	☒	MxSecurity	SECURITY_CODE	SECURITY_CODE_TYPE=US	SECURITY_CODE_DATE_END=29991231	R
97009		Security by Datastream c...	Security by Datastr...	Security by Datastre...	Security by Datastream ...	☒	MxSecurity	SECURITY_CODE_COMMUN	SECURITY_CODE_TYPE=DATASTR...	SECURITY_CODE_DATE_END=29991231	R
27044		Security by Datastream ID	Security by Datastr...	Security by Datastre...	Security by Datastream ID	☒	MxSecurity	SECURITY_CODE	SECURITY_CODE_TYPE=DATASTR...	SECURITY_CODE_DATE_END=29991231	R
25947		Security by Decalog ID	Security by Decalo...	Security by Decalog ID	Security by Decalog ID	☒	MxSecurity	SECURITY_CODE	SECURITY_CODE_TYPE=DECALOG	SECURITY_CODE_DATE_END=29991231	R
97667		Security by Europef ID	Security by Europe...	Security by Europef ID	Security by Europef ID	☒	MxSecurity	SECURITY_CODE	SECURITY_CODE_TYPE=EUROPERF	SECURITY_CODE_DATE_END=29991231	R
24318		Security by Factset code	Security by Factse...	Security by Factset c...	Security by Factset code	☒	MxSecurity	SECURITY_CODE_COMMUN	SECURITY_CODE_TYPE=FACTSET	SECURITY_CODE_DATE_END=29991231	R
31320		Security by Finalm ID	Security by Finalm ID	Security by Finalm ID	Security by Finalm ID	☒	MxSecurity	SECURITY_CODE	SECURITY_CODE_TYPE=77	SECURITY_CODE_DATE_END=29991231	R

Figure 19 – Using the “free 4” field in a research

Export

To export on the current date, specify the end date: “12/31/2999”.

SECURITY_CODE

[Code end date, ISIN / ISO]

Code end date

converted

Codification system

ISIN / ISO

Code end date

31/12/2999

Codification system

ISIN / ISO

Identifier

ID_ISIN

IF*

SUBIF*

IFNOT*

ELSE*

ENDIF*

Figure 20 - Exporting on the current date

For a history export, you must specify the date of validity of the research.

It will be converted in SQL into: “Start date <= date of integration and end date > date of integration”.

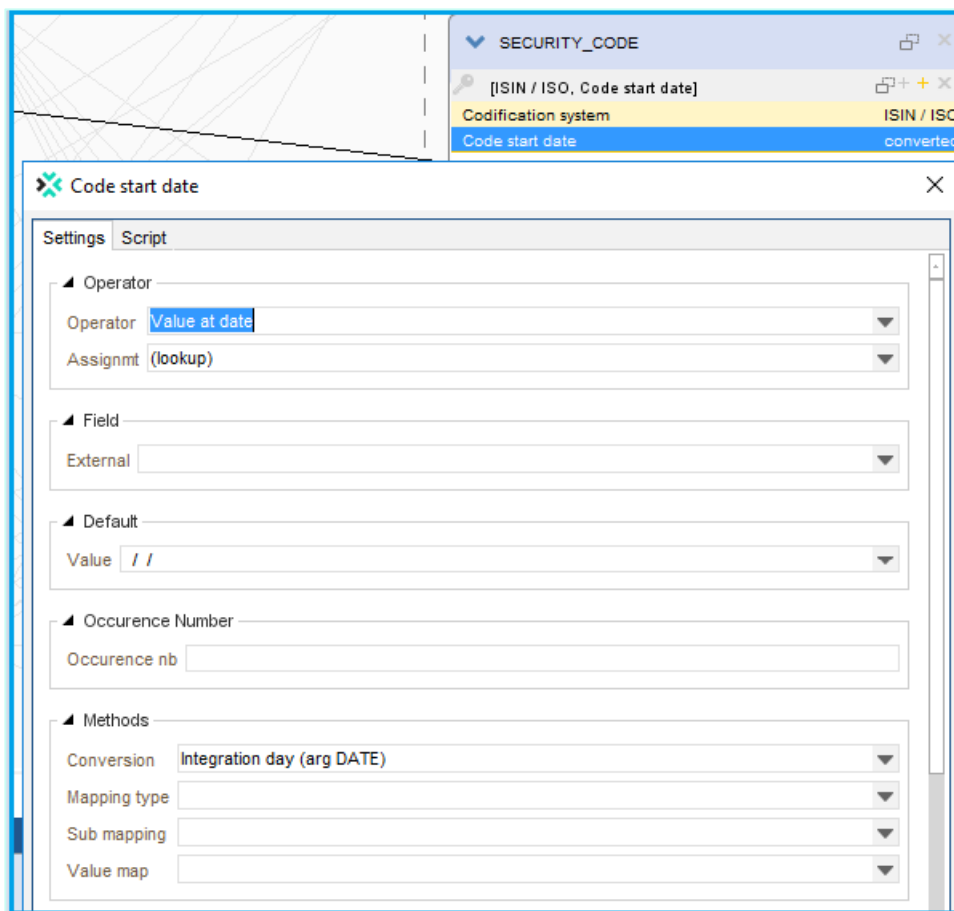


Figure 21 - Specifying the validity date of a research for a history export

Uniqueness Control

A specific unique mode is available to ensure that for a period, an identifier is used only once according to the key fields.

Below is an example for the class security.

Général Fields Catégories Versioning Validation Override Optimization Packaging Archive Misc.						
Fields						
Table	Key	Name	Unique	Precision	Scale	#Digits
▶ SECURITY						
▶ SECURITY_ACCRINT						
▶ SECURITY_AGENT						
▶ SECURITY_ANALYTICS						
▶ SECURITY_BARRIER						
▶ SECURITY_BOND						
▶ SECURITY_CDS						
▶ SECURITY_CDS_DEFAULT						
▶ SECURITY_CLASSIFICATION						
▶ SECURITY_CLIENTS						
▶ SECURITY_CODE						
SECURITY_CODE	🔑	SECURITY_CODE_SUBTYPE				
SECURITY_CODE	🔑	SECURITY_CODE_CURRENCY				
SECURITY_CODE	🔑	SECURITY_CODE_COUNTRY				
SECURITY_CODE	🔑	SECURITY_CODE_EXCHANGE				
SECURITY_CODE	🔑	SECURITY_CODE_DATE				
SECURITY_CODE	🔑	SECURITY_CODE_TYPE				
SECURITY_CODE		SECURITY_CODE	Unique - External, with Same Keys and in Same Time Interval			
SECURITY_CODE		SECURITY_CODE_COMMUN				
SECURITY_CODE		SECURITY_CODE_DATE_END				
SECURITY_CODE		SECURITY_CODE_IS_GROUP_CODE				
▶ SECURITY_COMMENT						
▶ SECURITY_COMMODITY						
▶ SECURITY_COMPONENT						
▶ SECURITY_CROSSCURRENCY						
▶ SECURITY_CURRENCY						
▶ SECURITY_DERIVATE						
▶ SECURITY_EQUITY						
▶ SECURITY_EVENT						
▶ SECURITY_FEED						
▶ SECURITY_FEES						
▶ SECURITY_FINANCING						
▶ SECURITY_FOREX						
▶ SECURITY_FUND						
▶ SECURITY_GENERIC						
▶ SECURITY_HISTO						

Figure 22 – Ensuring that an identifier is used only once during a specific period

Time Series Historization

Object

NeoXam DataHub manages times series data to limit the volume of data manipulated by the user and the system. This limitation corresponds to the most common usages. When this feature is activated, data reported to users contains a limited depth of history.

A combo box automatically appears on the detail screen; use that box to select the history depth to be displayed. By default, the history depth is set on Default.

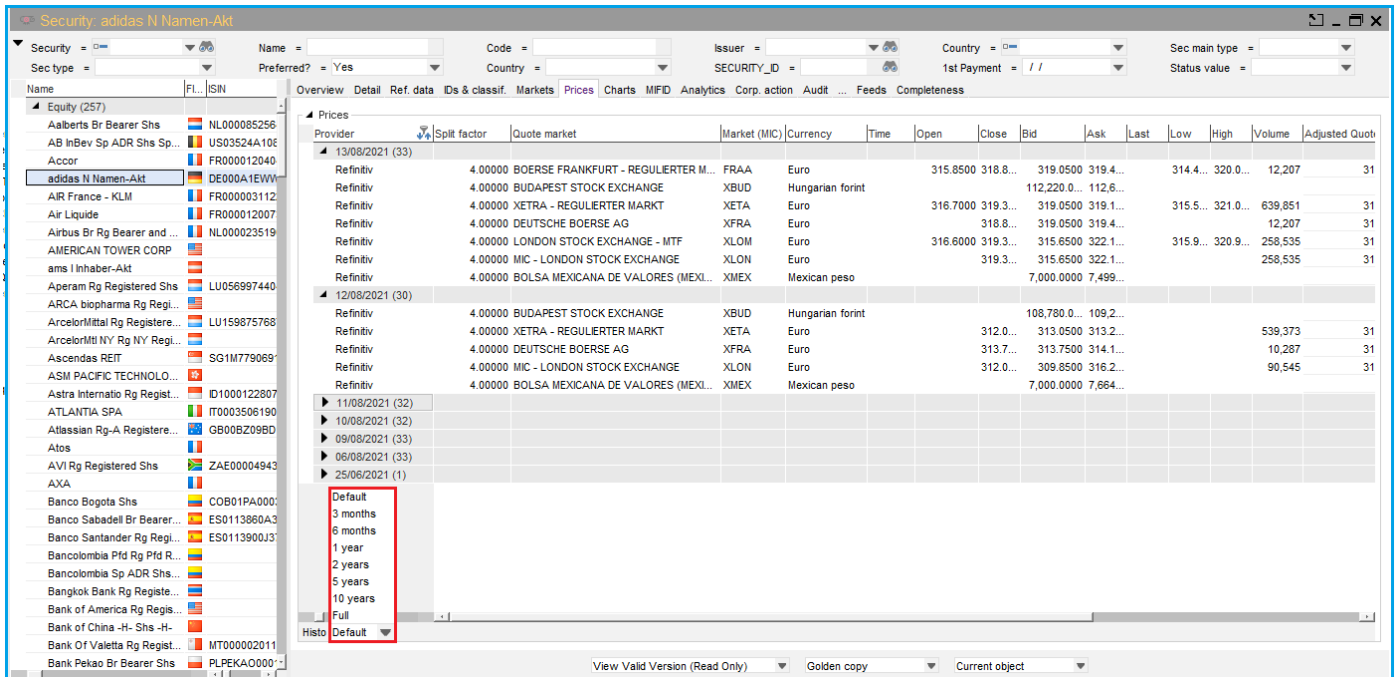


Figure 23 - The history depth combo box

Configuration

The eligible table must contain a dedicated field of the "DATE" type defined as a key. In the example below, QUOTE_DATE is the relevant date field.

SECURITY_QUOTE	Key level 5	QUOTE_CURRENCY	currency	quote currency	Integer...	SECURITY
SECURITY_QUOTE	Key level 4	QUOTE_TIME	time	time	8 String	
SECURITY_QUOTE	Key level 3	QUOTE_SOURCE	pricing source	pricing source	Integer...	SMARTLIST_ITEM Pricing Source
SECURITY_QUOTE	Key level 2	QUOTE_PROVIDER	provider	quote provider	Integer...	SMARTLIST_ITEM Feed
SECURITY_QUOTE	Key level 1	QUOTE_DATE	date	quote date	Date	

Figure 24 - Defining "DATE" type as a key

Important: A technical Date field with a name like "... _DATE_END must be created in the concerned table. This date field is NeoXam internal use only; it allows managing a validity date to chain records among themselves.

You may not be allowed to change it; the field appears as immutable in the system.

Finally, a default depth is defined for each table (i.e., D-7); if not defined, the totality of the table is reported.

If any of these elements is missing, the history management does not work.

The example below illustrates time series tables where history is managed.

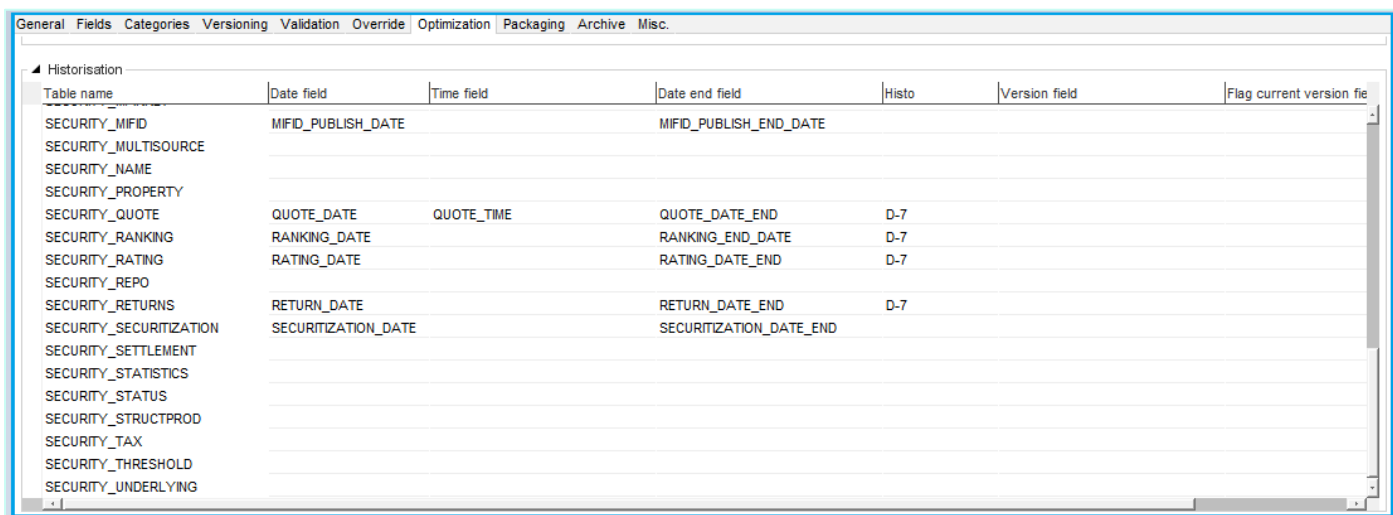


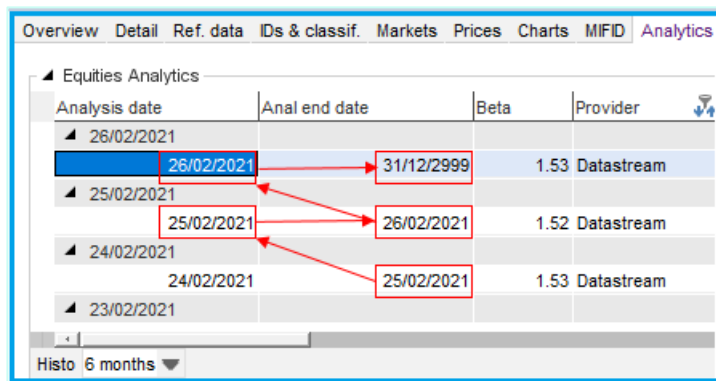
Table name	Date field	Time field	Date end field	Histo	Version field	Flag current version fie
SECURITY_MIFID	MIFID_PUBLISH_DATE		MIFID_PUBLISH_END_DATE			
SECURITY_MULTISOURCE						
SECURITY_NAME						
SECURITY_PROPERTY						
SECURITY_QUOTE	QUOTE_DATE	QUOTE_TIME	QUOTE_DATE_END	D-7		
SECURITY_RANKING	RANKING_DATE		RANKING_END_DATE	D-7		
SECURITY_RATING	RATING_DATE		RATING_DATE_END	D-7		
SECURITY_REPO						
SECURITY_RETURNS	RETURN_DATE		RETURN_DATE_END	D-7		
SECURITY_SECURITIZATION	SECURITIZATION_DATE		SECURITIZATION_DATE_END			
SECURITY_SETTLEMENT						
SECURITY_STATISTICS						
SECURITY_STATUS						
SECURITY_STRUCTPROD						
SECURITY_TAX						
SECURITY_THRESHOLD						
SECURITY_UNDERLYING						

Figure 25 - Time series tables with history management

The example below shows how the "end date" field chains records between themselves.

Note that:

1. The chaining always requires a combination key; here: date + provider.
2. The end date of the most recent records is always 31/12/2999.



The screenshot shows the 'Equities Analytics' section of the NeoXam interface. It displays a table with columns: Analysis date, Anal end date, Beta, and Provider. The table contains several rows of data, with red arrows indicating the chaining of records through the 'End Date' field. For example, the record for 26/02/2021 has an end date of 31/12/2999, which points to the next record for 25/02/2021. The table also includes a 'Histo' dropdown set to '6 months'.

Analysis date	Anal end date	Beta	Provider
26/02/2021	31/12/2999	1.53	Datastream
25/02/2021	26/02/2021	1.52	Datastream
24/02/2021	25/02/2021	1.53	Datastream
23/02/2021			

Figure 26 - Chaining records through the "End Date" field

History Discontinuity

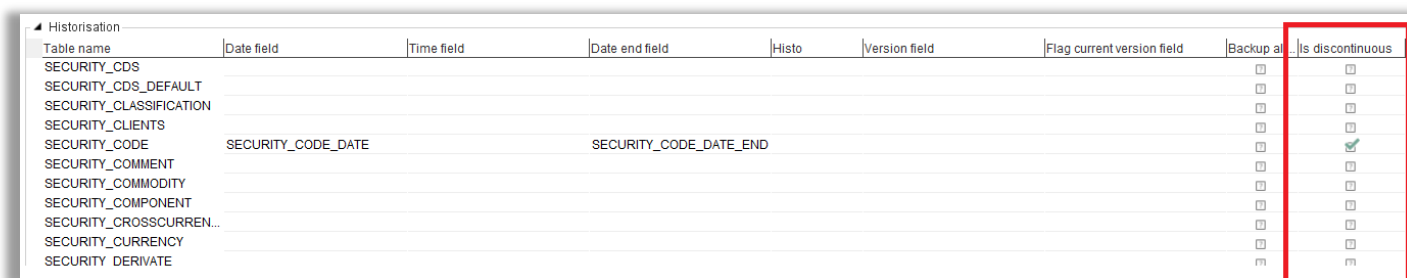
On some occasions, you may need to break the records date chaining and allow discontinuity in the record history. Discontinuity may be needed for:

Ratings: A rating agency may give a given grade to a security for a period of time. Then no grade at all because the security is not fulfilling anymore the minimum requirements to get a grade. Then, once the security can be graded again, give a new grade.

Identifiers: A type of identifier (for instance ISIN) may be attributed to a given security for a period of time, and then suspended, before being attributed again.

To deactivate the record history continuity:

1. Go to the class screen: Core Admin>Data Dictionary
2. From the Historisation section of the Optimization tab, select the **"Is discontinuous"** checkbox of the record that needs to be discontinued.



The screenshot shows the 'Historisation' section of the NeoXam interface. It displays a table with columns: Table name, Date field, Time field, Date end field, Histo, Version field, Flag current version field, Backup at, and Is discontinuous. The table contains several rows of data, with the 'Is discontinuous' checkbox checked for the 'SECURITY_CODE' record.

Table name	Date field	Time field	Date end field	Histo	Version field	Flag current version field	Backup at	Is discontinuous
SECURITY_CDS								☐
SECURITY_CDS_DEFAULT								☐
SECURITY_CLASSIFICATION								☐
SECURITY_CLIENTS								☐
SECURITY_CODE	SECURITY_CODE_DATE		SECURITY_CODE_DATE_END					☒
SECURITY_COMMENT								☐
SECURITY_COMMODITY								☐
SECURITY_COMPONENT								☐
SECURITY_CROSSCURREN...								☐
SECURITY_CURRENCY								☐
SECURITY_DERIVATE								☐

Figure 27 – Activating History Discontinuity on a record

3. You can now specify an end date that is not automatically converted by the system and create discontinuity on a record.

Example: Let's take the AGENT_RATING secondary table as an example.

When the **Is discontinued** check box of the record date is active, you can enter or update an end or start date, even when it creates a discontinuity. Dates are not recalculated by the system to chain and discontinuity is permitted. Here, there is a one-month time span between the AAA-rating that ends on February 1st, 2022 and the AAA rating that starts on March 1st, 2022.

Historic				
Provider	Type	Start Date	End Date	Rating
Manual	S&P LT	01/03/2022	31/12/2999	AAA
Manual	S&P LT	01/03/2021	01/02/2022	AAA-

Figure 28 – S&P rating showing record discontinuity

Data Migration

Important: If data exists in the tables before the configuration, it is mandatory to launch a service that computes the complete data chaining.

The procedure is the following:

1. In the execution environment, check you have a direct access to the database. The service will not pass through the NeoXam DataHub Server but connect to the database via the JDBC driver.
2. Shutdown the NeoXam DataHub server.
3. Execute the following command to launch the re-computation:

```
adm checkhisto "-cfg=mysql.cfg" "-table=TABLE"
```

with options: [-table=tablename] [-check] [-v]

- The "-table=TABLE%" option manages all the tables whose name starts with "TABLE". The table name is optional; by default, the service will manage all the history-managed tables.
- The "-check" option tests whether tables require re-computation.

```
adm checkhisto "-cfg=mysql.cfg" "-table=TABLE" -check
```

Note: With this option, the command can be launched when the server is running, as it is totally passive.

- Restart the SDH Server

Intraday Data

NeoXam DataHub allows storing intraday data series. This is particularly interesting with prices for organizations that want to store multiple intraday snapshots in addition to the daily EOD prices.

NeoXam DataHub manages this feature in the history.

Data Dictionary

A new field contains the time data:

- Name: user free (e.g., QUOTE_TIME)
- Type: String
- Length: 8 characters
- Key: Included in the unique key of the related table (i.e., SECURITY_QUOTE).

Example:

SECURITY_QUOTE						
SECURITY_QUOTE	.QUOTE_CURRENCY	Currency	Quote currency		Integer / Reference	SECURITY
SECURITY_QUOTE	.QUOTE_TIME	Time	Time		String	8
SECURITY_QUOTE	.QUOTE_PROVIDER	Provider	Quote provider		Integer / Reference	SMARTLIST_ITEM
SECURITY_QUOTE	.QUOTE_MARKET	Quote market	Quote market		Integer / Reference	EXCHANGE
SECURITY_QUOTE	.QUOTE_DATE	Date	Quote date		Date	

Figure 29 - Time data in a new field

This field contains:

- « Null » for EOD records (open / close price)
- Time of snapshot, otherwise (format: HH :MM :SS)

Note: In addition to this intraday specification, the "multiple time zones support" requires the creation of a timestamp field. See the [Time Zone](#) chapter.

History Configuration

The time field (i.e., QUOTE_TIME) must be declared in the history tab of the data dictionary screen.

Example:

Table name	Date field	Time field	Date end field	Histo	Version field	Flag current version field	Backu
SECURITY_MIFID	MIFID_PUBLISH_DATE		MIFID_PUBLISH_END_DATE				
SECURITY_MULTISOURCE							
SECURITY_NAME							
SECURITY_PROPERTY							
SECURITY_QUOTE	QUOTE_DATE	QUOTE_TIME	QUOTE_DATE_END	D-7			
SECURITY_RANKING	RANKING_DATE		RANKING_END_DATE	D-7			
SECURITY_RATING	RATING_DATE		RATING_DATE_END	D-7			
SECURITY_REPO							
SECURITY_RETURNS	RETURN_DATE		RETURN_DATE_END	D-7			
SECURITY_SECURITIZATION	SECURITIZATION_DATE		SECURITIZATION_DATE_END				
SECURITY_SETTLEMENT							
SECURITY_STATISTICS							
SECURITY_STATUS							
SECURITY_STRUCTPROD							
SECURITY_TAX							
SECURITY_THRESHOLD							
SECURITY_UNDERLYING							

Figure 30 – Declaring the time field in the history tab of the data dictionary

Time Zone

Depending on the requirement, three fields can be used to manage the date and field:

1. **Date field:** contains the current date (official date - 48hours).
2. **Time field:** contains the user time (official local time). This type of field should not be used in a multi-zone environment.
3. **Datetime field:** Date converted according to the time zone defined.

The "datetime" field allows saving a local time to a GMT time.

This conversion is done by the java platform itself. The “datetime” fields (and this type only) are converted according to the local and remote time zone during serialization and deserialization.

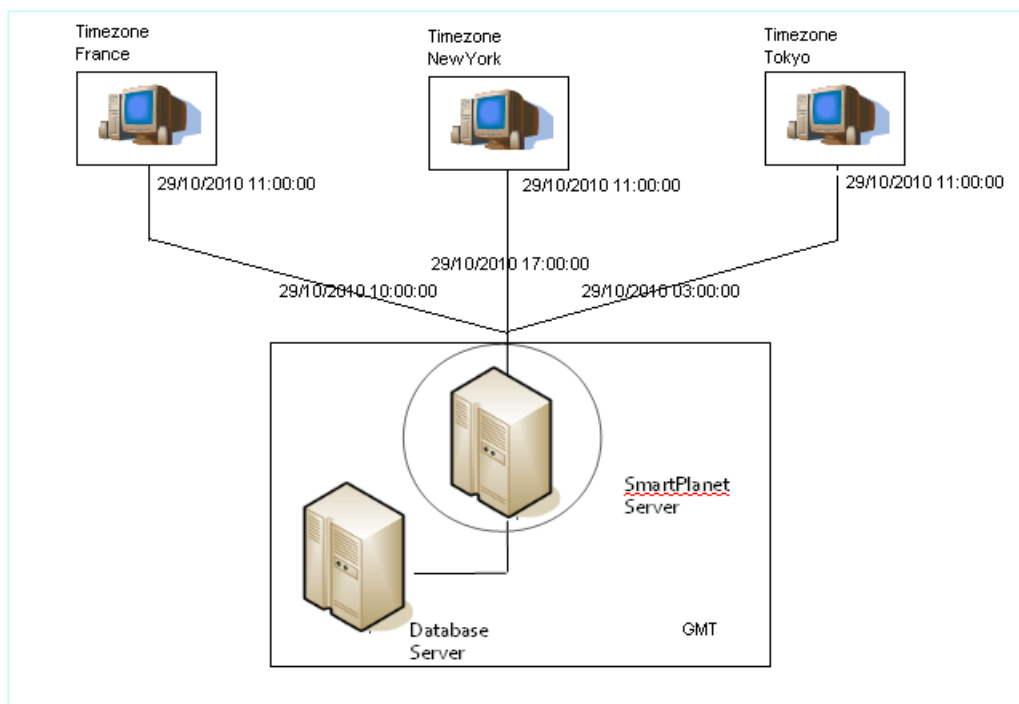


Figure 31 - Serialization and deserialization for time conversion

Displaying the “datetime” Field (delete or move down)

The “datetime” field is displayed using the user date format + HH:mm:ss format; it can appear in grids and as a single entry:

Provider	Date time	Quote market	Market (MIC)	Open	Close	Ask	Bid
▲ 25/02/2021 (29)							
Golden copy	25/02/2020 07:00:00			293.1000	293.1000	292.4000	292.3000
Golden copy	24/02/2020 22:00:00			325.9700	325.9700	323.1590	322.9390
Golden copy				362.6600	356.3200		
Golden copy					7,160.0000	7,240.0000	7,160.0000
Golden copy						106,100.0000	105,660.0000
Six Financial	25/02/2020 07:00:00	CBOE EUROPE -BXE ORDER BOOKS	BATE		293.1000		
Six Financial	24/02/2020 22:00:00	BX World	BRX	325.9700	325.9700	323.1590	322.9390
Six Financial	25/02/2020 07:00:00	CBOE EUROPE - CXE ORDER BOOKS	CHIX		293.6500		294.2000
Six Financial	25/02/2020 07:00:00	MIC - EUROTIX	ETIX		292.5000	292.4000	292.3000

Figure 32 - Displaying the “datetime” field

If you want to edit this kind of field, the usual calendar is proposed, with an extra entry field that shows the time.

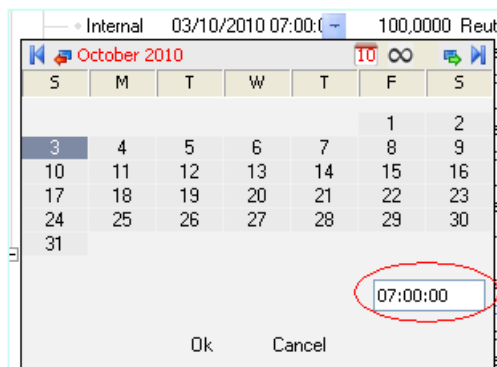


Figure 33 - Editing the "datetime" field

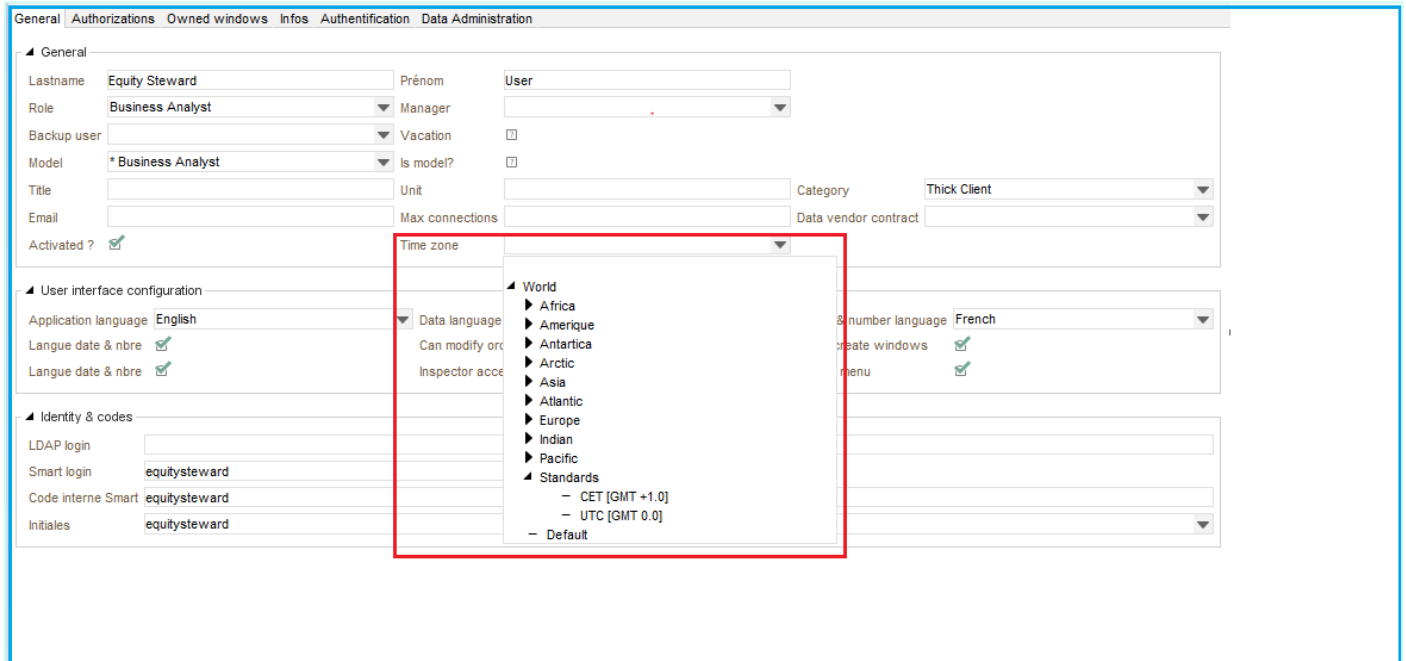
Configuration

Client Time Zone Setting

The time zone is configured by the user (MxUser); if no time zone has been configured or no user is declared (Server), the time zone of the machine is taken.

The time zone is configurable on a detail screen for users.

Time zones have been organized by geographical location as follows (MxList TIMEZONE_LIST):



The screenshot shows the 'General' tab of the NeoXam user configuration interface. The 'Time zone' dropdown menu is open, showing a list of world time zones. The user 'Equity Steward' is configured with role 'Business Analyst' and category 'Thick Client'.

Figure 34 - Time zones in the "General" tab

FeedProc settings

If the client knows that the "datetime" field in the integration file is converted for a specific time zone, he can use the TIMEZONE property to correctly convert the datetime before it is sent to the server.

The value used by the property corresponds to the values present in the FREE 1column of the TIMEZONE_LIST list.

Warning: When jobs and scheduler tasks are defined, a user other than the scheduler can be specified; the user specified MUST HAVE the same time zone as the scheduler. The only way to manage different time zones is to use the TIMEZONE property in the feedproc.

Specific use case: Historicized data

NeoXam DataHub manages historicized data at the time level. The time is currently stored as a string; to manage time zone several modifications must be made.

Let's take, for instance, price integration. Prices are stored in the SECURITY_QUOTE table. The date is stored in the QUOTE_DATE field and the time in QUOTE_TIME.

The transformation consists in adding a new field: QUOTE_DATETIME.

SECURITY_QUOTE	Key Level 6	QUOTE_DATETIME	Date time	Date time (time zone)		heur...	DateTime	
SECURITY_QUOTE	Key Level 5	QUOTE_CURRENCY	Currency	Quote currency		devis...	Integer / Reference	SECURITY
SECURITY_QUOTE	Key Level 4	QUOTE_TIME	Time	Time		heure	String	8
SECURITY_QUOTE	Key Level 3	QUOTE_PROVIDER	Provider	Quote provider		fourn...	Integer / Reference	SMARTLIST_7
SECURITY_QUOTE	Key Level 2	QUOTE_MARKET	Quote market	Quote market		Place...	Integer / Reference	EXCHANGE
SECURITY_QUOTE	Key Level 1	QUOTE_DATE	Date	Quote date		date ...	Date	

Figure 35 - The "QUOTE_DATETIME" field

The historicization settings must be updated to take the new field into account:

Main	Fields	Categories	Audit trail	Versioning	Validation	Optimization	Packaging	Archive	Misc.
► Memory cache									
► Database tuning									
▼ Historisation									
Table name	Date field	Time field	Date end field	Histo					
SECURITY_GENERIC									
SECURITY_HISTO									
SECURITY_INDEX									
SECURITY_INDEX_SOURCE									
SECURITY_INTEREST									
SECURITY_MARKET									
SECURITY_MIFID	MIFID_PUBLISH_DATE		MIFID_PUBLISH_END_DATE						
SECURITY_MULTISOURCE									
SECURITY_NAME									
SECURITY_QUOTE	QUOTE_DATE	QUOTE_DATE TIME	QUOTE_DATE_END	D-7					
SECURITY_RANKING	RANKING_DATE		RANKING_END_DATE	D-7					
SECURITY_RATING	RATING_DATE		RATING_DATE_END	D-7					

Figure 36 - Updating the historicization settings

Note: Data migration is not described here; we assume the database is empty.

The example below illustrates a 48-hour day, each user has simultaneously captured the same quotation, at different times in different places: Newman in NYC, Dupont in Paris and Yamashi in Tokyo.

Grid						
	Provider	Date time	Close	User	Quote end date	Date
07/02/2011						
	Reference		101,0000	EOD	31/12/2999	07/02/2011
	Internal		101,0000	EOD	31/12/2999	07/02/2011
	Internal	08/02/2011 07:00:00	100,0000	Newman	07/02/2011	07/02/2011
	Internal	07/02/2011 22:00:00	100,0000	Dupont	07/02/2011	07/02/2011
	Internal	07/02/2011 21:00:00	99,0000	Newman	07/02/2011	07/02/2011
	Internal	07/02/2011 14:00:00	100,0000	Yamashi	07/02/2011	07/02/2011
	Internal	07/02/2011 12:00:00	99,0000	Dupont	07/02/2011	07/02/2011
	Internal	07/02/2011 11:00:00	98,0000	Newman	07/02/2011	07/02/2011
	Internal	07/02/2011 04:00:00	99,0000	Yamashi	07/02/2011	07/02/2011
	Internal	07/02/2011 02:00:00	98,0000	Dupont	07/02/2011	07/02/2011
	Internal	06/02/2011 18:00:00	98,0000	Yamashi	07/02/2011	07/02/2011
06/02/2011						
	Reference		91,0000	EOD	07/02/2011	06/02/2011
	Internal		91,0000	EOD	07/02/2011	06/02/2011
	Internal	07/02/2011 07:00:00	90,0000	Newman	06/02/2011	06/02/2011
	Internal	06/02/2011 22:00:00	90,0000	Dupont	06/02/2011	06/02/2011
	Internal	06/02/2011 21:00:00	89,0000	Newman	06/02/2011	06/02/2011
	Internal	06/02/2011 14:00:00	90,0000	Yamashi	06/02/2011	06/02/2011
	Internal	06/02/2011 12:00:00	89,0000	Dupont	06/02/2011	06/02/2011
	Internal	06/02/2011 11:00:00	88,0000	Newman	06/02/2011	06/02/2011
	Internal	06/02/2011 04:00:00	89,0000	Yamashi	06/02/2011	06/02/2011
	Internal	06/02/2011 02:00:00	88,0000	Dupont	06/02/2011	06/02/2011
	Internal	05/02/2011 18:00:00	88,0000	Yamashi	06/02/2011	06/02/2011

Figure 37 - Same quotation captured at different times in different places

Database Time Zone Settings

The database time zone is by default the machine time zone; however, you can change this. The conversion between the database time zone and the NeoXam DataHub server time zone is handled by the JDBC driver.

Below are the parameters for the standard database used by NeoXam DataHub.

Oracle

To change the database time zone, use the SET TIME_ZONE clause of the ALTER DATABASE statement. For example:

```
ALTER DATABASE SET TIME_ZONE='Europe/London';
```

Ref:

http://download.oracle.com/docs/cd/B19306_01/server.102/b14225/ch4datetime.htm#i1006705

SQL Server

SQL Server takes the time zone of the machine on which it is installed.

MySQL

You can set the system time zone for MySQL Server at startup with the option

[`--timezone=timezone\_name`](#)

Ref: <http://dev.mysql.com/doc/refman/5.1/en/time-zone-support.html>

Date Time and Time Zone Manipulation

Mapping

Currently, the only way to map a date time field is to use a script combining 2 fields. Here is an example of a script that combines a date and a time field:

```
strDate := getValueInImportArray(row, "C");
strTime := getValueInImportArray(row, "B");
if(strTime = "") return "";
curDate := getDate(strDate, "yyyyMMdd")
return fmtDate(curDate, "dd/MM/yyyy") + " " + strTime;
```

Business Rules

Several functions have been implemented in order to manage the date time and time zone:

- `getDateWithTime()`: Returns a date that includes the time (time argument format: HH:mm:ss)
- `applyTimezone()`: Returns a date according to the time zone defined in argument. The time zone value given in argument must correspond to a value of the FREE 1 column of the TIMEZONE_LIST list.
- `getTimeOfDate()`: Returns the time on a given date (return format: HH:mm:ss).
- `getHours()`: Returns the hour on a given date.
- `getMinutes()`: Return the minutes on a given date.
- `getSeconds()`: Returns the seconds on a given date.

Data Migration

When clients with an existing database want to use "datetime" fields, they need to perform data migration.

To perform this migration, a new service has been implemented: `DatetimeMigrationToolService`.
Classpath: `lib.system.services.DateTimeMigrationToolService`

This service allows combining the date and time field into a datetime field:

- 6 arguments: [class][table] [date][time][date time technical field name][date time field name]: fill one field according to the date and time field mentioned.
- All fields must necessarily belong to the same table.
- "bulksize", an optional argument can be added after the required arguments.
- Data is migrated if both the date and time fields are filled.

Ex: MxSecurity SECURITY_QUOTE QUOTE_DATE QUOTE_TIME DATE_TIME DateTime
500000

The service must run on the server side as it deals with database.

Important: The server must be restarted once the migration has been performed.

Technical Parameters

PARAMETER	DESC.	POSSIBLE VALUES
system.db.stringfields.nosizecheck	Size of fields listed in the value of this parameter is not checked by the system when the user changes it. Listed fields must be part of <i>MxClassDef</i> and must be of STRING type.	The syntax is as follows: TABLENAME.FIELDNAME separated by a comma For example: SMARTCLASS_FIELD.FIELD_NAME,SMARTCLASS_TABLE.TABLE_NAME. This example allows defining a size greater than 255 for the two fields listed above.

Installation

Time Zone

New Field type: Datetime

To use the "datetime" field, the SYSTEM_FIELD_TYPE list must be updated with the new date time field type (using the right upgrade package).

Values
Definition details
Categories

Definitions

Name (En.)Field typeName (Ch.)字段类型

Name (Fr.)Type de champ

CodeSYSTEM_FIELD_TYPECategoryCore

TypeType Complement

PrivacyNeoXamFeed

Custom Sort ☐Not Hierarchical Sort ☐

Free Flag ☐

Values

ID

197

NX #	Sort	Label - English	Label - French	Label - Chinese	Internal Code	Active	Free 1	Free 2	Free 3	Free 4	Item Privacy	Icon	Comment
3577	Integer / Reference	Entier / Référence	整数/参考	1		☒					NeoXam		
3578	Decimal	Décimal	小数	2		☒					NeoXam		
3579	String	Chaîne de caractères	字符串	3		☒					NeoXam		
3580	Date	Date	日期	4		☒					NeoXam		
3581	Text	Texte	文本	5		☒					NeoXam		
21871	File	Fichier	文件	6		☒					NeoXam		
253468	DateTime	DateTime	日期和时间	8		☒					NeoXam		
395227	Document	Document	文档	9		☒					NeoXam		

Figure 38 - Updating the "SYSTEM_FIELD_TYPE" list

Warning: Once the new type is created, you need to restart the server to take the new field type into account. When the server is restarted, the new field is ready to be used. Be aware that after a "date time" field has been created, you cannot launch a server in a version lower than 3.1.

In this configuration, the "datetime" field is converted in the local time zone (client and server). The client time zone can be user-dependent. To manage the time zone at a user-level several settings must be created.

Data Dictionary Update

To manage the time zone at a user-level, the MxUser class must be updated with a new TIMEZONE field in the adequate upgrade package.

General Fields Categories Versioning Validation Override Optimization Packaging Archive Misc.						
Fields						
Table	Key	Name	Type	Size	Reference	List
SMARTUSER		MANAGER	Integer / Reference		SMARTUSER	
SMARTUSER		MAX_CONNECTIONS	Integer / Reference			
SMARTUSER		MENU_ORDERING	Text			
SMARTUSER		MONITOR_CONSUMPTION	Integer / Reference		SMARTLIST_ITEM	Yes/No
SMARTUSER		NAME	String	40		
SMARTUSER		PASSWORD	String	20		
SMARTUSER		PRIVACY_LEVEL	Integer / Reference		SMARTLIST_ITEM	Privacy Levels
SMARTUSER		ROLE_TYPE	Integer / Reference		SMARTLIST_ITEM	User Roles
SMARTUSER		SMARTCODE	String	50		
SMARTUSER		TIMEZONE	Integer / Reference		SMARTLIST_ITEM	Time Zones
SMARTUSER		USER_CALENDAR	Integer / Reference		CALENDAR	
SMARTUSER		USER_CATEGORY	Integer / Reference		SMARTLIST_ITEM	User Categories
SMARTUSER		USER_MODEL	Integer / Reference		SMARTUSER	
SMARTUSER		VACATION_FLAG	Integer / Reference		SMARTLIST_ITEM	Yes/No

Figure 39 - Updating the MxUser class

This field is referencing a new list Timezone list which must be created; it is provided in the installation package. It contains all the time zones the java platform can handle.

Code Historicization

Data Dictionary: Import Mapping MxExternalFeedMap

General Fields Categories Versioning Validation Override Optimization Packaging Archive Misc.							
Fields							
Table	Key	Name	Type	Size	Reference	List	Secondary refe
▸ XFEEDMAP							
▸ XFEEDMAP_ALTMAP							
▸ XFEEDMAP_INTERNAL							
▸ XFEEDMAP_ITEM							
XFEEDMAP_ITEM		ASSIGNMENT_OPERATOR	Integer / Reference		SMARTLIST_ITEM	Comparison Operators	
XFEEDMAP_ITEM		ASSIGNMENT_TYPE	Integer / Reference		SMARTLIST_ITEM	External Feed - Assignment Types	
XFEEDMAP_ITEM		CONVERSION_SCRIPT	Text				
XFEEDMAP_ITEM		CONVERSION_TYPE	Integer / Reference		SMARTLIST_ITEM		
XFEEDMAP_ITEM		DEFAULT_VALUE	String	25			
XFEEDMAP_ITEM		EXTERNAL_FIELD	Integer / Reference		SMARTLIST_ITEM		
XFEEDMAP_ITEM		EXTERNAL_FIELD_OCC	String	255			
XFEEDMAP_ITEM		IF_LOGICAL_OPERATOR	Integer / Reference		SMARTLIST_ITEM	Operators	
XFEEDMAP_ITEM		IF_VALUE	String	100			
XFEEDMAP_ITEM		INTERNAL_FIELD	Integer / Reference		SMARTCLASS_FIELD		
XFEEDMAP_ITEM		ITEM_ACTIVE	Integer / Reference		SMARTLIST_ITEM	Yes/No	
XFEEDMAP_ITEM		ITEM_COMMENT	Text				
XFEEDMAP_ITEM		ITEM_IF_LEVEL	Integer / Reference				
XFEEDMAP_ITEM		ITEM_INTERNAL_CODE	String	30			
XFEEDMAP_ITEM		ITEM_NODE	Integer / Reference		SMARTTREE_NODE		
XFEEDMAP_ITEM		ITEM_ORDER	Integer / Reference				
XFEEDMAP_ITEM		ITEM_SUBMAP	Integer / Reference		XFEEDMAP_SUBMAP		
XFEEDMAP_ITEM		XFEEDMAP	Integer / Reference		XFEEDMAP		
XFEEDMAP_ITEM		XVALUEMAP	Integer / Reference		XVALUEMAP		
XFEEDMAP_ITEM		XVALUEMAPPING_TYPE	Integer / Reference		SMARTLIST_ITEM	External feed - Standard mappings	
▸ XFEEDMAP_ITEM_EXT...							
▸ XFEEDMAP_LABEL							
▸ XFEEDMAP_SUBMAP							

Figure 40 – Import Mapping; the "ASSIGNMENT_OPERATOR" field for using operators such as ">" and "<".

The ASSIGNMENT_OPERATOR field allows creating criteria in the “lookup” or “key lookup” with new operators such as “>”, “<”.

By default, the operator is “equal”.

Data Dictionary: Export mapping MxOutPutFeedMap

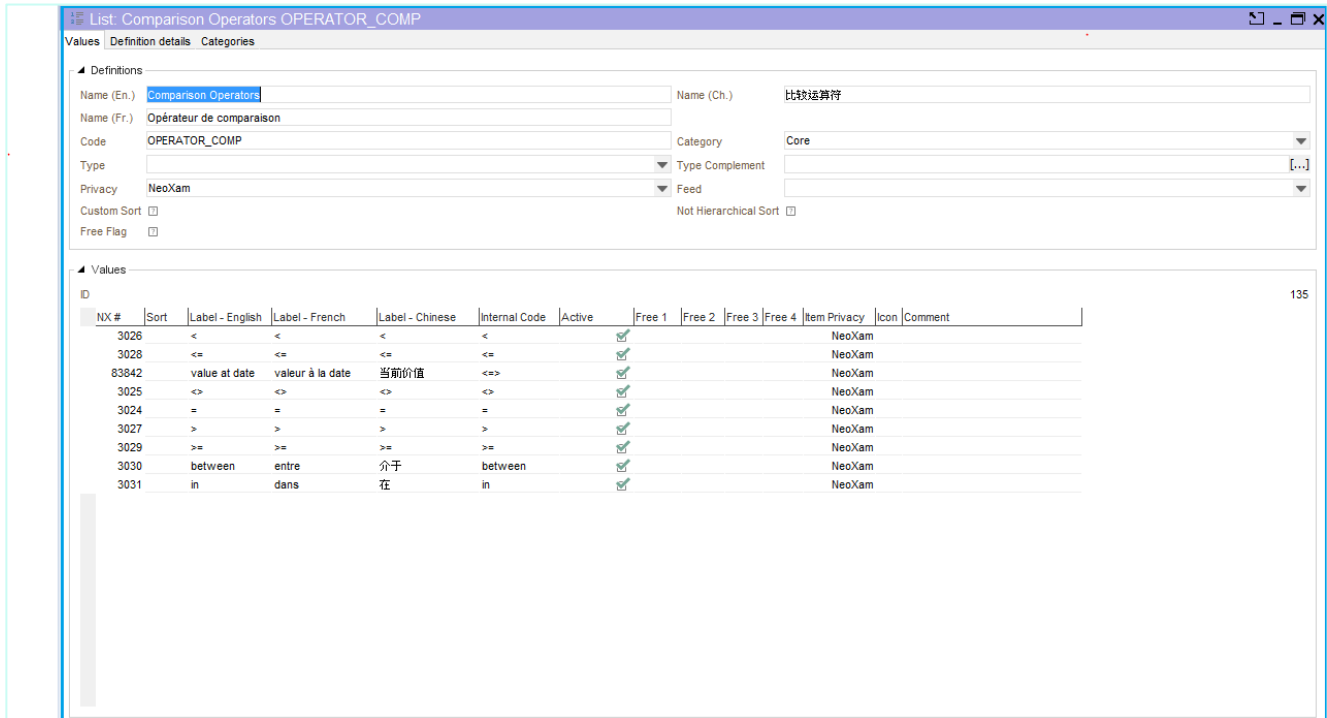
Table	Key	Name	Nom court (angl.)	Nom long (angl.)	...	...	...	Type	Size	Reference	List
► OUTFEEDMAP											
▲ OUTFEEDMAP_ITEM											
OUTFEEDMAP_ITEM	Key Level 2	ITEM_ORDER	order	item order	...	...	...	Integer / Reference			
OUTFEEDMAP_ITEM	Key Level 1	ITEM_SUBMAP	submap	submap	...	...	...	Integer / Reference		OUTFEEDMAP_SUBMAP	
OUTFEEDMAP_ITEM		ASSIGNMENT_OPERATOR	operator	operator	...	...	...	Integer / Reference		SMARTLIST_ITEM	Comparison Operators
OUTFEEDMAP_ITEM		ASSIGNMENT_TYPE	assignmt	assignment type	...	...	...	Integer / Reference		SMARTLIST_ITEM	Output Feed - Assignment...
OUTFEEDMAP_ITEM		CONVERSION_TYPE	conversion	conversion type	...	...	...	Integer / Reference		SMARTLIST_ITEM	
OUTFEEDMAP_ITEM		DEFAULT_VALUE	value	default value	...	...	...	String	25		
OUTFEEDMAP_ITEM		EXTERNAL_FIELD	external	external field	...	...	...	Integer / Reference		SMARTLIST_ITEM	
OUTFEEDMAP_ITEM		INTERNAL_FIELD	internal	internal field	...	...	...	Integer / Reference		SMARTCLASS_FIELD	
OUTFEEDMAP_ITEM		ITEM_ACTIVE	active ?	active item	...	...	...	Integer / Reference		SMARTLIST_ITEM	Yes/No
OUTFEEDMAP_ITEM		ITEM_COMMENT	Comment	Comment	...	...	...	Text			
OUTFEEDMAP_ITEM		ITEM_INTERNAL_CODE	code	code	...	...	...	String	25		
OUTFEEDMAP_ITEM		ITEM_NODE	node	item node	...	...	...	Integer / Reference		SMARTTREE_NODE	
OUTFEEDMAP_ITEM		ITEM_SCRIPT	script	script	...	...	...	Text			
OUTFEEDMAP_ITEM		ITEM_TRIGGER	trigger	trigger	...	...	...	Integer / Reference		OUTFEEDMAP_TRIGGER	
OUTFEEDMAP_ITEM		OUTPUT_CONFIGURATION	configuration	configuration	...	...	...	Text			
OUTFEEDMAP_ITEM		OUTVALUEMAP	value map	value map	...	...	...	Integer / Reference		OUTVALUEMAP	
OUTFEEDMAP_ITEM		VALUEMAPPING_TYPE	mapping type	mapping type	...	...	...	Integer / Reference		SMARTLIST_ITEM	External feed - Standard ...

Figure 41 - Export Mapping; the "ASSIGNMENT_OPERATOR" field for using operators such as ">" and "<".

Use the "ASSIGNMENT_OPERATOR" field to create criteria in your "lookup" with new operators such as ">" and "<".

For default case, the operator is "equal".

List: Defining Operators



List: Comparison Operators OPERATOR_COMP

Values | Definition details | Categories

Definitions

Name (En.) Name (Ch.)

Name (Fr.)

Code Category

Type Type Complement

Privacy Feed

Custom Sort ☐ Not Hierarchical Sort ☐

Free Flag ☐

Values

ID	NX #	Sort	Label - English	Label - French	Label - Chinese	Internal Code	Active	Free 1	Free 2	Free 3	Free 4	Item Privacy	Icon	Comment
	3026	<	<	<	<		☒					NeoXam		
	3028	<=	<=	<=	<=		☒					NeoXam		
	83842	value at date	valeur à la date	当前价值	<=>		☒					NeoXam		
	3025	<>	<>	<>	<>		☒					NeoXam		
	3024	=	=	=	=		☒					NeoXam		
	3027	>	>	>	>		☒					NeoXam		
	3029	>=	>=	>=	>=		☒					NeoXam		
	3030	between	entre	介于	between		☒					NeoXam		
	3031	in	dans	在	in		☒					NeoXam		

Figure 42 - Defining operators

This operator is converted in SQL into: "Start date <= value and end date > value".

Data Masking

Introduction to Data Masking

Data masking or data obfuscation is the process of hiding original data with random characters or data.

The main reason for applying masking to a data field is to protect data that is classified as personal identifiable data, personal sensitive data, or commercially sensitive data; however, the data must remain usable for the purposes of undertaking valid test cycles. It must also look real and appear consistent. It is more common to have masking applied to data that is represented outside of a corporate production system, in other words where data is needed for the purpose of application development, building program extensions, and conducting various test cycles. It is common practice in enterprise computing to take data from the production systems to fill the data component, required for these non-production environments. However, the practice is not always restricted to non-production environments. In some organizations, data that appears on terminal

screens to call center operators may have masking dynamically applied based on user security permissions. (e.g.: Preventing call center operators from viewing Credit Card Numbers in billing systems).

The data masking can be done as:

- **Static Data Masking:** by creating a copy of original data base with masked values. This requires additional data base space, and an asynchronous creation process can generate inconsistency between masked and original data.
- **Dynamic Data Masking:** by keeping the original database and adding an additional layer to apply data masking functions.

This section describes the Dynamic Data Masking feature existing in DataHub:

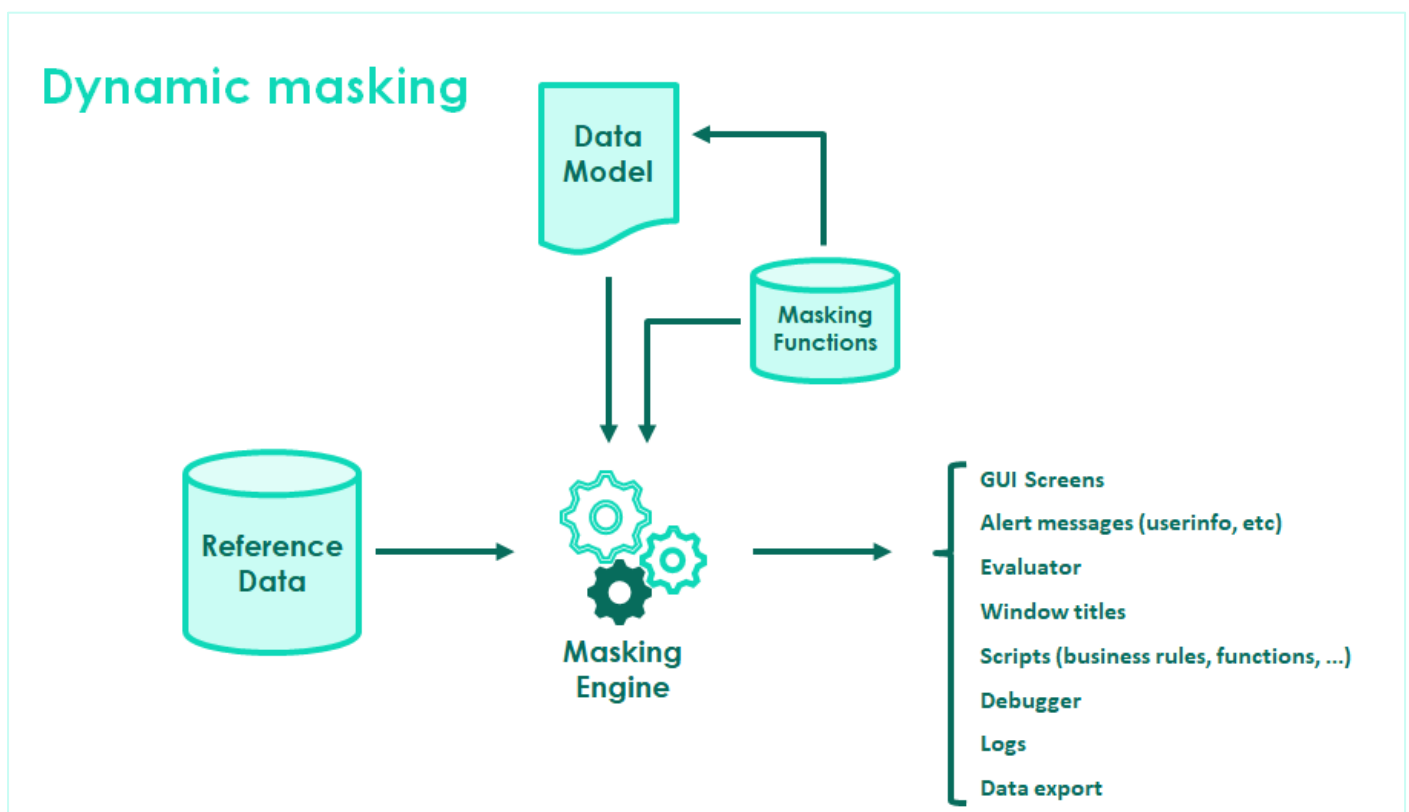


Figure 43 - Dynamic Masking

Configuring Data Masking

Masking Functions

Masking functions can be created to mask the data in database so that data display is restricted to specific users.

The functions can be defined either in Java code or Business Rules (user defined). They can have a default parameter used to guide the behavior of function.

Masking Functions

Function	Data Format	Internal Code	Class/Rule	Description	Input	If Input Empty?	Parameters	Output
String - Replace	String	STRING.REPLACE	lib.common.client.masking.ReplaceString	Replaces all characters of a string with the character provided as parameter. If no parameter or a wrong format is provided, the default masking parameter must be used.	Air France	Output empty	Example: x Hard-coded Default: *	xxxxxx xxxx
Date - Replace	Date	DATE.REPLACE	lib.common.client.masking.ReplaceDate	Replaces the date with the date provided as parameter.	15/05/2001	Output empty	Example: Hard-coded Default: 31/12/2199	31/12/2199

Datetime - Replace	Datetime	DATETIME.REPLACE	lib.common.client.masking.ReplaceDatetime	Replaces the date and time with the date provided as parameter.	15/05/2001	Output empty	Example: Hard-coded Default: 31/12/2199 00:00:00	31/12/2199 00:00:00
Decimal - Replace	Decimal	DECIMAL.REPLACE	lib.common.client.masking.ReplaceDecimal	Replaces all digits of decimal with the digit provided as parameter.	150,99	Output empty	Example: 9 Hard-coded Default: 9	999,99
Integer - Replace	Integer	INTEGER.REPLACE	lib.common.client.masking.ReplaceInteger	Replaces all digits of integer with the digit provided as parameter.	150000	Output empty	Example: 9 Hard-coded Default: 9	999999
Reference - Replace	Reference	REFERENCE.REPLACE	lib.common.client.masking.ReplaceReference	Returns an empty object, null or a generic string (i.e., "****")	Object	Output null	Example: "—", "****", null Hard-coded Default: ***	***
Text - Replace	Text	TEXT.REPLACE	lib.common.client.masking.ReplaceText	Replaces all characters of a text with the character provided as parameter	Air France	Output empty	Replace with: x Hard-coded Default: *	xxxxxx xxxx

Functions must be created for a specific type of data (Integer, Decimal, String, etc.) and are referenced in the **FIELD_MASKING_FUNCTIONS** list, as shown on the following screenshot:

List: Masking Functions FIELD_MASKING_FUNCTIONS

values

Definition details

Categories

Definitions

Name (En.)

Masking Functions

Name (Ch.)

Name (Fr.)

[...]

Code

FIELD_MASKING_FUNCTIONS

Category

Type

Hierarchical

Type Complement

Privacy

NeoXam

Feed

Custom Sort

☐

Not Hierarchical Sort

Free Flag

☐

Values

ID

NX #	Sort	Label - English	Label - French	...	Internal Code	Active	RULE_CLASS	Free 2	Free 3	Free 4	Item Privac
608847	01	String			STRING	☒					NeoX
608850	01.01	String - Replace			STRING.REPLACE	☒	lib.common.masking.ReplaceString	*			NeoX
608851	01.02	String - Replace betw...			STRING.REPLACEBETW...	☒	lib.common.masking.ReplaceBetweenString				NeoX
608848	02	Integer			INTEGER	☒					NeoX
608852	02.01	Integer - Replace			INTEGER.REPLACE	☒	lib.common.masking.ReplaceInteger	9			NeoX
608903	02.02	Integer - Replace Rule			INTEGER.REPLACERULE	☒	REPLACE_INTEGER	9			NeoX
608893	03	Date			DATE	☒					NeoX
608895	03.01	Date - Replace			DATE.REPLACE	☒	lib.common.masking.ReplaceDate	31/12/2199			NeoX
608894	04	Text			TEXT	☒					NeoX
608896	04.01	Text - Replace			TEXT.REPLACE	☒	lib.common.masking.ReplaceText	*			NeoX
608897	05	Decimal			DECIMAL	☒					NeoX
608900	05.01	Decimal - Replace			DECIMAL.REPLACE	☒	lib.common.masking.ReplaceDecimal	9			NeoX
608898	06	Datetime			DATETIME	☒					NeoX
608901	06.01	Datetime - Replace			DATETIME.REPLACE	☒	lib.common.masking.ReplaceDatetime	31/12/2199 00:00:00			NeoX
608899	07	Reference			REFERENCE	☒					NeoX
608902	07.01	Reference - Replace			REFERENCE.REPLACE	☒	lib.common.masking.ReplaceReference	***			NeoX

Figure 44 - The Masking Functions

1. The Internal Code is composed as **FIELD_TYPE.FUNCTION_CODE**.

- The **FIELD_TYPE** is mapped into field types as follows:

FIELD_TYPE	Field Type
STRING	String
INTEGER	Integer/Reference
DATE	Date
TEXT	Text
DECIMAL	Decimal
DATETIME	Datetime
REFERENCE	Integer/Reference

FILE	File
DOCUMENT	Document

- When the field type is an Integer/Reference, there are two possibilities:
 - **No reference field** ("Reference" column empty in data dictionary), in this case only INTEGER functions are available.
 - **Reference field** ("Reference" column populated in data dictionary), in this case only REFERENCE functions are available.
- 2. The **RULE_CLASS** refers to a business rule if a function is a user-defined business rule or to an external coded module (Java code). The available masking functions are available in the [Masking Functions](#) paragraph.
- 3. Default function parameter

If the mask function is defined as a Business Rule, the following arguments are received when the rule is called:

Data Format	Arguments
String	Input:= Value to be masked replacementParam:= Parameter received as argument from configuration (user level or data dictionary level) defaultReplacementParam:= Default parameter received as argument from configuration (Free2 field)
Integer	Input:= Value to be masked replacementParam:= Parameter received as argument from configuration (user level or data dictionary level) defaultReplacementParam:= Default parameter received as argument from configuration (Free2 field)
Date	Input:= Value to be masked replacementParam:= Parameter received as argument from configuration (user level or data dictionary level) defaultReplacementParam:= Default parameter received as argument from configuration (Free2 field)
Text	Input:= Value to be masked replacementParam:= Parameter received as argument from configuration (user level or data dictionary level)

	defaultReplacementParam:= Default parameter received as argument from configuration (Free2 field)
Decimal	Input:= Value to be masked replacementParam:= Parameter received as argument from configuration (user level or data dictionary level) defaultReplacementParam:= Default parameter received as argument from configuration (Free2 field)
Datetime	Input:= Value to be masked replacementParam:= Parameter received as argument from configuration (user level or data dictionary level) defaultReplacementParam:= Default parameter received as argument from configuration (Free2 field)
Reference - List	Input:= Value to be masked replacementParam:= Parameter received as argument from configuration (user level or data dictionary level) defaultReplacementParam:= Default parameter received as argument from configuration (Free2 field)
Reference - Object	Input:= Value to be masked replacementParam:= Parameter received as argument from configuration (user level or data dictionary level) defaultReplacementParam:= Default parameter received as argument from configuration (Free2 field)
File	
Document	

The return value of the function will be the value considered as the masked output of the function.

User-Level Restrictions

Once the functions are defined, they can be used to define restrictions on how data is displayed to users.

A masking function can be defined for an entire class, for a secondary table or for a field (additionally, a rec spec can be associated with it).

To set up a mask for a specific field, the function must be defined for a Class as follows:

General Authorizations Owned windows Infos Authentication Data Administration									
Authorization view									
Class	Auth type	Object	Table	Field	Condition	Record filter	Mask Function	Mask Parameters	
Agent	Full Access								
Agent	Read (Mask)		AGENT_NAME	SHORT_NAME			String - Replace	*	
Agent	Read (Mask)		AGENT_NAME	NAME			String - Replace	*	
Audit trail	No Access								
Class definition	Full Access								
Corporate action	Full Access								
Corporate action	Skip Validation								
Country									
EDM Project	Read								
EDM Project	Write		EDM_PROJECT_...						
Feed proc	Full Access								
Input feed map	Full Access								

Figure 45 - Setting up a mask

1. The "Authorization Type" must be set as "Read (mask)".
2. "Table" and "Field" must be selected.
3. A function must be selected in "Mask Function".
4. If needed, a "Mask Parameter" can be entered.

Class-Level Restrictions

You can also define masking functions at the field-level in the "Data Dictionary" configuration:

Général Fields Catégories Versioning Validation Override Optimization Packaging Archive Misc.									
Fields									
Table	Key	Name	Display	Virtual	Script	Mask Function	Mask Parameters	Field privacy	Data group
SECURITY		ACCRINT_ROUNDING_MODE						Public	Static Data
SECURITY		AGREEMENT_TYPE						Public	Static Data
SECURITY		AMORTIZATION_R332_19						Public	Static Data
SECURITY		ANNONCE_DATE						Public	Static Data
SECURITY		CFI_CODE				String - Replace	*	Public	
SECURITY		CIC_LISTED_COUNTRY						Public	Static Data
SECURITY		CIC_TYPE						Public	Static Data
SECURITY		COLLATERAL_DESCRIPTION						Public	
SECURITY		COMMERCIAL_COUNTRY						Public	Static Data
SECURITY		COUNTRY						Public	Static Data
SECURITY		CREATION_DATE						Public	Static Data
SECURITY		CREATION_SOURCE						Public	Static Data
SECURITY		CURRENCY						Public	Static Data
SECURITY		CURRENCY_REPORTING						NeoXam	
SECURITY		DELIVERY_DATE						Public	Static Data
SECURITY		DELIVERY_MODE						Public	Static Data
SECURITY		DEMAND_LINK						Public	Static Data
SECURITY		EUROCLEAR_ACCEPT_DATE						Public	Static Data
SECURITY		EXCHANGE_MARKET						Public	Static Data
SECURITY		EXCHANGE_TAX						Public	Static Data
SECURITY		EXDIVIDEND_DELAY						Public	Static Data
SECURITY		EXDIVIDEND_STATUS						Public	Static Data
SECURITY		EXPOSURE_COUNTRY						Public	Static Data
SECURITY		FIRST_PAYMENT_DATE						Public	Static Data
SECURITY		FISN						Public	Static Data

Figure 46 - Defining masking functions at the field-level

1. Select a function in the "Mask Function" column.
2. If needed, a "Mask Parameter" can be entered.

Once this is done, you can have a masking defined for an entire class or a specific table.

General Authorizations Owned windows Infos Authentication Data Administration									
Authorization view									
Class	Auth type	Object	Table	Field	Condition	Record filter	Mask Function	Mask Parameters	
Input value map	Skip Validation								
▲ List									
List	Full Access								
▲ Output feed map									
Output feed map	Full Access								
▲ Portfolio									
Portfolio	Full Access								
Portfolio	Skip Validation								
▲ Rule									
Rule	Full Access								
▲ Security									
Security	Full Access								
Security	Read (Mask)								
▲ Transaction									
Transaction	Full Access								
Transaction	Skip Validation								
▲ User									
User	Read								
User	Execute								

Figure 47 - A masking function applied to a class or a table

1. The "Authorization Type" must be set as "Read (mask)".
2. The "Table" can also be selected.
3. A function cannot be selected in "Mask Function" (and neither a "Mask Parameter").

In this case, all the fields of the "Security" class will be masked for the user by means of the functions defined in the "Data Dictionary".

Any update on the masking functions is immediately sent to client sessions; therefore, sessions do not require to be refreshed.

Using Data Masking

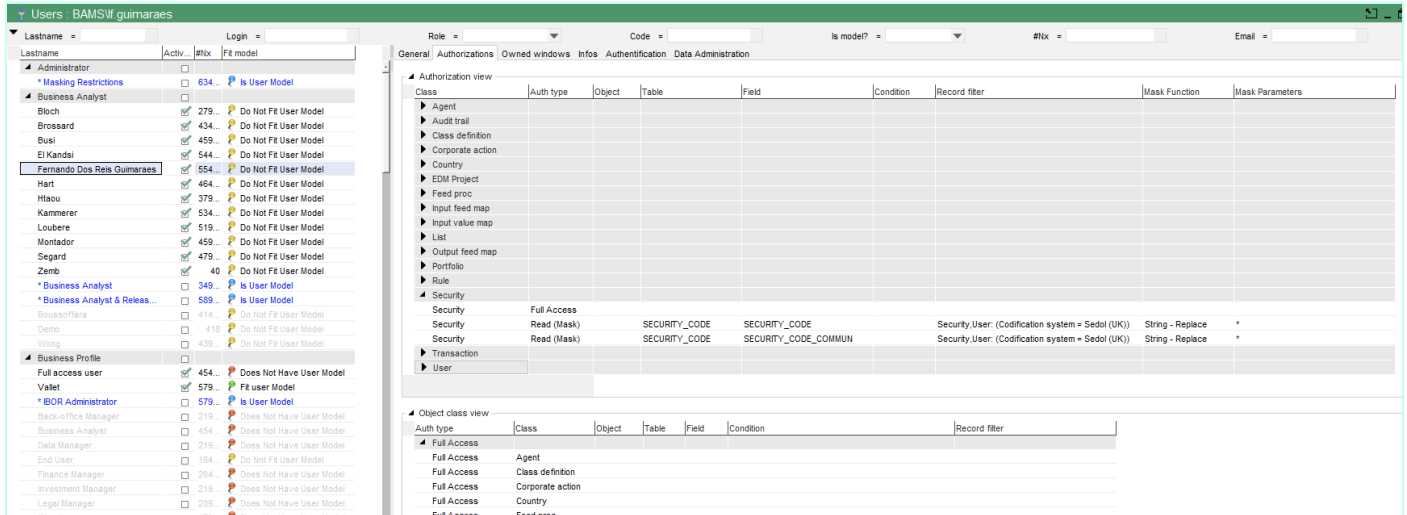
Masking Securities Sedol (UK) Codes for a Specific User

To configure a scenario where the Sedol (UK) code is masked for a specific user, the fields containing the Security Code must be masked when Security Code Type = Sedol (UK).

Steps to follow for configuration:

1. As an Admin user, log in to Data Hub.
2. Go to Menu > System Administration > Users and Activity.
3. Find the user to whom the restriction applies.
4. Go to the Authorizations tab.

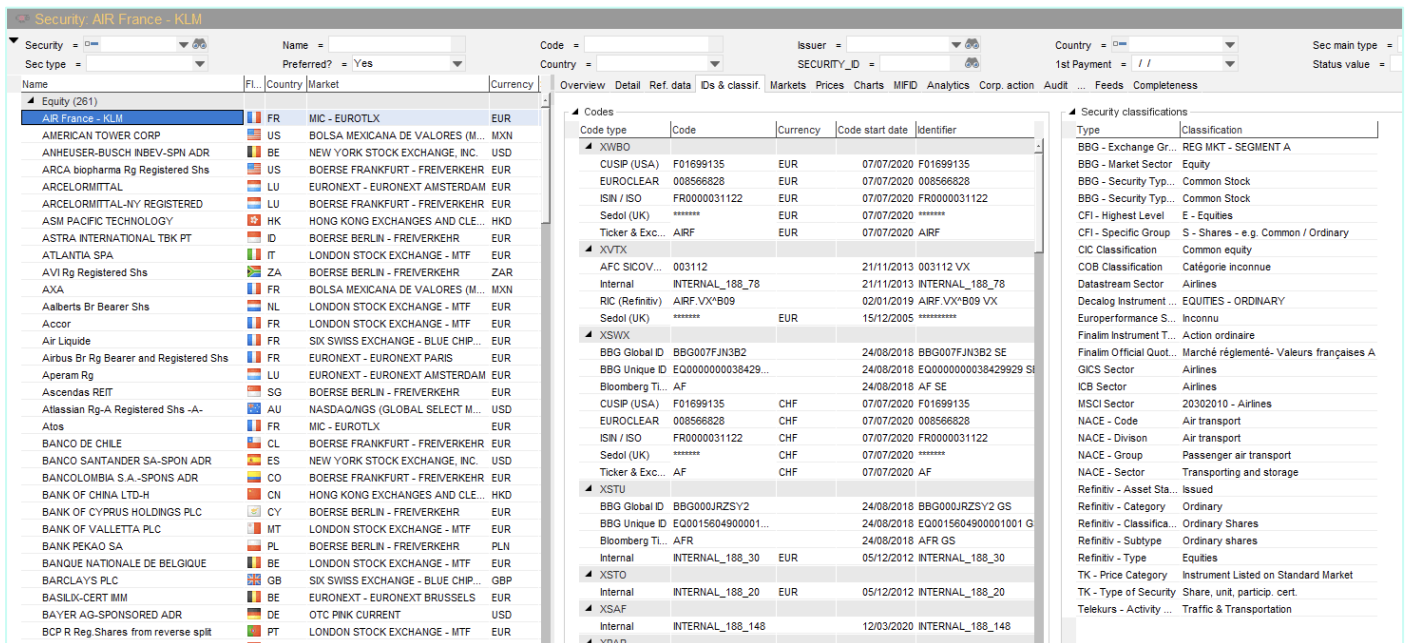
5. Add a new line as follows (and save):



Class	Auth type	Object	Table	Field	Condition	Record filter	Mask Function	Mask Parameters
Security	Full Access						String - Replace	*

Figure 48 - Masking the Sedol (UK) codes of Securities for a specific user (1)

Expected results:



Code type	Code	Currency	Code start date	Identifier
Sedol (UK)	*****	EUR	07/07/2020	*****

Figure 49 - Masking the Sedol (UK) codes of Securities for a specific user (2)

Masking Interfaces

Currently, the results of the masking functions are active in:

1. Query and Detail screens,
2. Audit Screens,
3. File exports.

Masking Interfaces - Query and Detail Screens

The results of masking rules are visible on any screen elements referring to masked fields.

In the image below, we can see that the ISIN code and issuer are masked for the current user on the selected security:

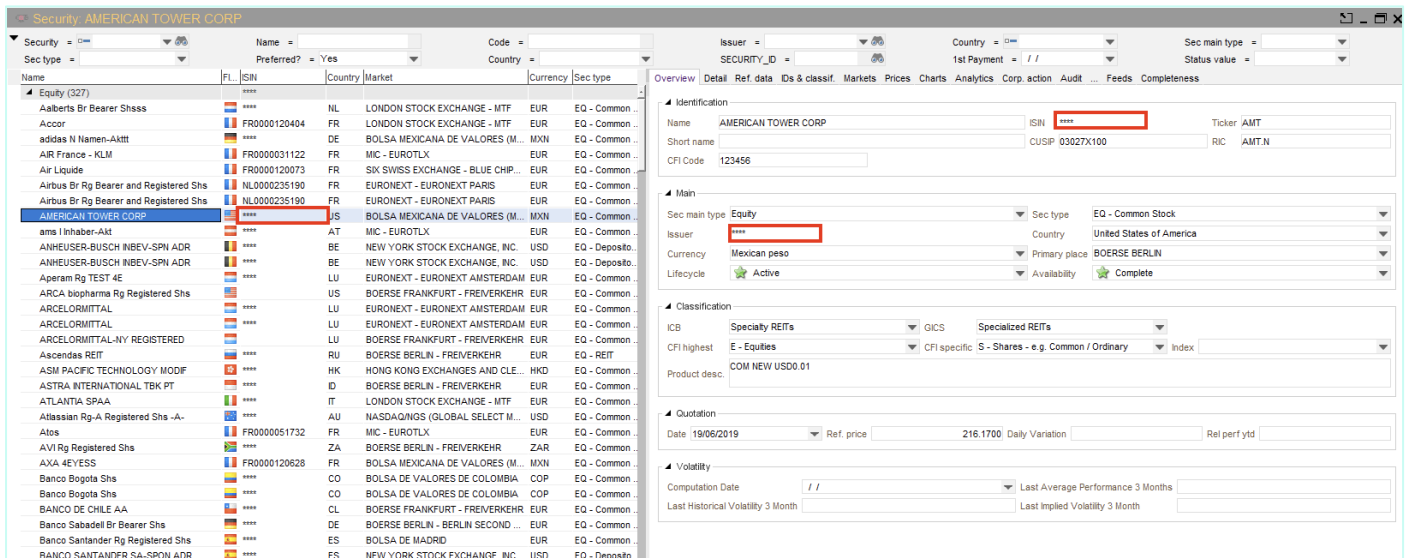


Figure 50 - Masking Rules: Results

If a masked field is used as Query research criteria, the masked values are not considered.

Data Masking and Scripting Languages

Data Masking supports scripting languages.

To identify masked fields and values when a script is being built, the following methods are available with the **MxUser** object class:

- **MxUser.hasMask(obj, table, field, [recspec])** returns **1** if a masking function exists for a user or at least one value found; returns **0** if no value is found.

- ***MxUser.hasMaskForRecord(obj, table, field, recnum)***: returns **1** if a masking function exists for a user; returns **0** if no masking function exists.
- ***MxUser.getMaskedValue(obj, table, field, [recspec])***: returns a bidimensional array of values, containing pairs (recnum, val), where val is:
 - **-1** for a record field not masked,
 - The masked value for a masked field according to the predefined masking function configured in the system.
- ***MxUser.getMaskedValueForRecord(obj, table, field, recnum)***: returns the masked value of the field according to the predefined masking function configured on system. Returns **-1** if a masking function is not existing to the field in that record.

The example below shows how a field is masked to a user and what is the masked value.

```

1 actualUser := actualuser()
2 isSedolMasked := actualUser.hasMask(obj1, "SECURITY_CODE", "SECURITY_CODE", "SECURITY_CODE_TYPE='GB'")
3
4 if(isSedolMasked>0){
5
6   sedolCodeMasked := actualUser.getMaskedValue(obj1, "SECURITY_CODE", "SECURITY_CODE", "SECURITY_CODE_TYPE='GB'")
7   sedolCodeMasked := sedolCodeMasked[0]
8   sedolCodeMasked := sedolCodeMasked[1]
9   userinfo("Sedol(UK) code is masked as " + sedolCodeMasked)
10
11 }else{
12   sedolCode := obj1.SECURITY_CODE[SECURITY_CODE_TYPE='GB']
13   userinfo("Sedol(UK) code is " + sedolCode)
14 }
15
16
17
18
19
20
21

```

Figure 51 - Masked field and masked value in a script